

[DRL][FR] Final Proposal

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pptx

簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 ****final project proposal****

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明, 也把三個附檔都過了一遍。我的判斷很明確：

最適合你拿來做這門 DRL final project proposal 的題目, 是：

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制 (Adaptive Traffic Signal Control)」。

你附的兩份 drone reference, 其實是在示範同一種教授喜歡的敘事方式：先把問題的重要性講大, 再整理 related work, 接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**, 還把資料量、模型、限制都量化寫出來；Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文, 但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本, 這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**；搭配你上傳的 mp4 內容, 我理解教授現在要的不是「先把系統全部做完」, 而是要你先把整條論文故事線講順：**這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚, 就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**；而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它**最貼近教授 reference 的味道**。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它**非常適合 A-to-Z 的 proposal 寫法**。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。📄 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方面，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based
- RL / DRL traffic signal control：Q-learning、DQN、PPO、multi-agent
- reward / state design：queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment：SUMO / SUMO-RL
- Agent：先用 DQN 或 PPO
- State：各車道 queue length、waiting time、current phase、phase elapsed time
- Action：切換到哪個 phase，或維持 / 切換
- Reward：負的總等待時間 + queue penalty
- Baselines：fixed-time、actuated control
- Metrics：average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。

因為它同時滿足四件事：

- 有**真實世界動機**
- 有**明確 baseline 與 metrics**
- 有**成熟工具鏈**
- 有**好寫的論文故事**

你決定選這題的話，我下一則就直接幫你寫成**教授格式的 final project proposal 初稿**，包含：
題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「**這個題目能不能當 final project proposal**」：可以。

如果你是問「**可不可以直接當 DRL 專案原封不動拿去做**」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。 [YouTube+2](#)

這個專案本身其實**很完整，也很有 proposal 故事性**：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。 [GitHub+1](#)

但如果你的課是 **Deep Reinforcement Learning**，那這個 repo **目前不是一個 DRL 專案**。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的設計痕跡；從結構看起來，它更像是**電腦視覺 + 多模態語音 + 邊緣裝置整合**的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。 [GitHub+3](#)

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」 [GitHub+1](#)

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- **DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance**
- **Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation**
- **A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired**

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。[知乎專欄+1](#)

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。[GitHub+2](#)

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你這個題目改寫成一份**符合教授 A-to-Z 架構的 proposal 題目 + Abstract + Introduction + Related Work + Proposed Design**。

另外如果是 MemPalace 這個 AI Agent 永久記憶開源專案呢 是否適合當作題目
且因為 MemPalace 的亮點就是能把 AI 的記憶 "本地化" 如果這個本地化模型還要跑重武器 DRL 有點本末倒置 (本地化就是要越輕量越好)
所以我想從「輕量化 DRL」的切入方向
不要用深度神經網絡 (Deep Q-Network)，改用 Multi-Armed Bandit (MAB, 多臂老虎機) 或 輕量級 Q-Learning

但我連 ai agent 的架設都不會 也沒用過 這樣的程度適合做這個題目嗎
不太適合的話是否有折衷方案 連 ai agent 概念都不用摸呢
顯示更多顯示較少

可以，但我會先給你一個很務實的判斷：

MemPalace 適合當靈感來源，也適合當研究對象；但不適合你現在這個程度直接做成「完整 AI agent 記憶系統實作」的 final project。

因為 MemPalace 本身不是小玩具，它是完整的 **local-first AI memory** 系統：用語意搜尋保留逐字記憶、預設後端是 ChromaDB、知識圖譜用本機 SQLite，還有 MCP 工具、hooks、agent diary、benchmark 等完整配套。官方 repo 也主打本地化、零 API、LongMemEval raw R@5 96.6%。[GitHub+2](#)

你抓的方向其實是對的：**如果題目核心價值是「本地化、輕量化、低依賴」**，那再往上疊一個需要重訓練、重推論的 DQN，確實很可能會讓題目失焦。MemPalace 官方自己也一直強調 bounded wake-up context，大約只載入 L0+L1 的 ~600-900 tokens，且搜尋是按需觸發，不是整包上下文全塞進去；這種設計哲學本來就偏向「資源節制」而不是「深模型硬壓」。這是你可以在 proposal 裡合理主張的切入點。[mempalaceofficial.com+2](#)

但要注意一個關鍵：**MAB 或 tabular Q-learning 並不等於 Deep RL。**

Contextual bandit 比較像是「每回合看到 context，選一個 action，只看到被選 action 的 reward」，是介於 supervised learning 與 RL 之間的問題；tabular Q-learning 則是完整 RL，但它假設你有可管理的有限 state/action，透過探索與更新表格來學策略。官方與教材都把 contextual bandit 和 Q-learning 定義得很清楚，所以如果你的課名是 *Deep Reinforcement Learning*，你若只做純 MAB，教授有機會質疑「這題不夠 deep」。[Microsoft GitHub+2](#)

所以我對你這題的建議不是「放棄 MemPalace」，而是：

最適合你的版本

不要做完整 agent。改做：

「本地記憶系統中的輕量化決策層」

也就是把 MemPalace 當成既有記憶後端，你的研究重點不是重造一個 agent，而是研究：

在 token budget、延遲限制、不同 query 類型下，
要用哪一種記憶存取策略最划算？

這樣你的題目就會很乾淨。

我最推薦的題目版本

題目 A：最穩、最適合你

A Contextual Bandit for Token-Budgeted Local Memory Retrieval

白話版就是：

每來一個 query，bandit 根據 context 決定要走哪條 retrieval 路線。

例如 action 可以是：

- 只用 wake-up
- 只用 search
- 先 wake-up 再 search
- 只查某個 wing
- 查 knowledge graph
- top-k 取 3 / 5 / 10

這些 action 都很小、很離散，很符合 contextual bandit 「選有限個 action、看 chosen action reward」的型態。MemPalace 官方也提供了不用 agent 的 CLI 與 Python API：你可以直接跑 `mempalace wake-up`、`mempalace search`，或用 `search_memories()` 之類的 Python API 把結果接進你自己的實驗程式，不一定要碰 MCP agent。🔗 mempalaceofficial.com+2

這題最大的優點是：

你完全可以不會架 AI agent，也能做。

第二順位：稍微更像 RL

題目 B：進階但還可控

Tabular Q-Learning for Multi-Step Local Memory Access Policies

這版把問題變成多步決策，例如：

1. 先讀 wake-up

2. 判斷要不要搜 wing
3. 搜不到再深搜
4. 最後決定回傳多少內容

這時候就不是 one-shot bandit，而是有狀態轉移的 sequential decision problem，tabular Q-learning 在概念上比較合理。Q-learning 本來就是不需要先知道完整 MDP，就能從互動資料學 action-value；而且 Gymnasium 也有官方 tabular Q-learning 教學與自訂環境入口，對學生來說比 DQN 友善很多。

 Dive into Deep

Learning+1

但這題比題目 A 稍難，因為你要自己定義 state transition。

你現在的程度，適不適合？

我的判斷是：

做「完整 MemPalace agent + 本地模型 + MCP + hooks + RL」：不適合。

MemPalace 官方文件已經很明白，它平常的使用方式會牽涉到 MCP integration、Claude Code plugin、Gemini CLI、hooks；而「specialist agents」目前官方 docs 主要描述的是透過 MCP tools 寫 diary 的 workflow，不是那種你開箱即用就有完整 agent orchestration 的教學專案。對一個還沒碰過 AI agent 架設的人，這個入口太陡。  mempalaceofficial.com+2

做「MemPalace 當後端 + 你自己寫一個輕量 decision layer」：適合。

因為你可以把所有 agent 概念先拿掉，只保留：

- query
- retrieval action
- reward
- evaluation

這樣本質上就是一個 machine learning / RL 實驗題，不是 agent engineering 題。

如果你想連 agent 概念都不要碰

那我建議你直接再退一步，做成：

題目 C：最保守、最容易成功

Lightweight RL for Local Memory Retrieval Strategy Selection

這題甚至可以不真的安裝 MemPalace。

你只要自己準備一個小型資料集，例如：

- 一批對話紀錄 / 筆記 / 文件片段
- 一組 query

- 每個 query 對應的 relevant memory 片段標註

然後模擬幾種 retrieval strategy：

- 全域搜尋
- 分 room 搜尋
- 限 top-k
- 先短上下文再深搜
- 查知識圖譜/metadata 或不查

最後用 contextual bandit 或 tabular Q-learning 去學「哪種 query 應該走哪種策略」，reward 可以是：

$$\text{reward} = \text{recall@k} - \alpha \cdot \text{token_cost} - \beta \cdot \text{latency}$$

這樣你：

- 不用會 agent
- 不用會 MCP
- 不用接本地 LLM
- 甚至不用碰 MemPalace 真正的工程堆疊

但論文故事仍然成立，而且非常貼近你在意的「本地化」「輕量化」「不要重武器」。這個方向也跟 contextual bandit 在互動式推薦、客服 agent 裡的實際應用邏輯很接近；Microsoft 就有把 contextual bandits / MAB 用在 virtual agent 的推薦與意圖消歧上，重點正是 partial feedback 與資料效率。  [arXiv+1](#)

我給你的最終建議

若以你現在的程度，我會這樣排序：

最推薦：

題目 A：Contextual Bandit for Token-Budgeted Local Memory Retrieval

因為最輕、最不需要 agent、最符合你要的本地化邏輯。

次推薦：

題目 C：完全不碰 agent 的 retrieval strategy selection

最保守，成功率最高。

不推薦現在做：

完整 MemPalace agent 系統 + RL 強化

因為你會先死在系統整合，不是死在研究本身。

一句話結論

MemPalace 可以做，但不要把題目定成「做一個 AI agent 永久記憶系統」。

對你最好的版本是把它改成：

「本地記憶系統中，如何用輕量化 RL / contextual bandit，在 token 與延遲限制下選擇最佳 retrieval 策略？」

這樣你幾乎可以完全不碰 agent 概念，卻還是保留 MemPalace 的亮點與研究價值。官方 GitHub 就是 [MemPalace/mempalace](#)。 

下一步最有幫助的做法，是我直接幫你把這題寫成教授 A-to-Z 架構的 final project proposal 題目 + abstract + 方法設計。

看完你的說明 我決定不要 MemPalace, 盲人眼鏡 這兩個題目了
所以我要請你再推薦我其他三個題目

好，我這次不把先前提過的交通號誌控制算進來，直接給你 3 個新的 DRL final project 題目。我挑題的標準是：

要能套進你教授那種 Motivation → Related Work → Proposed Method → Experiments → Findings 的 A-to-Z 結構、不用碰 AI agent 架設、有現成環境或官方工具可用，而且對你現在的程度不會太失控。

Gymnasium 提供標準 RL 環境介面與自訂環境教學，Stable-Baselines3 則有可靠的 DQN / PPO 實作，這代表你可以把重心放在「題目與實驗設計」，而不是先卡死在基礎框架。  [健身房文檔+5](#)

1. 用 DRL 做庫存補貨策略最佳化

我最推薦你把這題當成**第一順位**。原因很簡單：它最不需要碰 agent 概念，問題定義也最乾淨。你可以把它寫成「在需求波動下，agent 如何決定每期補貨量，以最小化缺貨成本、持有成本與訂購成本」。這種題目非常適合教授的 A-to-Z：動機很好講，related work 可以寫傳統 EOQ / heuristic / OR 方法，method 可以定義 state、action、reward，experiments 則直接和 base-stock policy、heuristic policy 或 OR baseline 比。OR-Gym 這條線本來就是把 knapsack、bin packing、multi-echelon inventory management 等經典 OR 問題重寫成 RL 任務，並且和 MILP、heuristics 做 benchmark；現在也有更新到 Gymnasium 相容的 inventory repo。  [arXiv+3](#)

如果你想控制難度，這題還能自然分層：

最簡版做**單倉、單品項、離散補貨量**；中階版做**多倉或 lead time**；進階版才碰 multi-echelon。你甚至可以先用 tabular Q-learning 做很小的 state space，再升級成 DQN 或 PPO，這樣既照顧你目前程度，也比較符合課程的 DRL 味道。對你來說，它最大的優點是：**實驗好跑、baseline 好找、失敗也容易 debug**。  [arXiv+2](#)

2. 用 DRL 做建築能源 / 電池排程控制

這題是我給你的**第二順位**，也是三題裡「最像當代應用論文」的一題。CityLearn 本來就是為建築能源管理、需求響應與多智能體 RL 研究建立的標準化環境，官方文件明講它的目的是讓不同 RL 演算法在同一套城市建築能源協調問題上可以被公平比較；相關論文也指出，城市建築與需求響應是 RL 很有代表性的真實應用場景。CityLearn v2 甚至擴充到更多 building control 問題與 benchmark。  [citylearn.net+3](#)

這題很適合教授的 proposal 架構，因為故事線很完整：

Motivation 可以講節能、尖峰削減、電網壓力；But 可以講 rule-based control 或 MPC 的侷限；Cure 就是 DRL；Experiments 可以比較 rule-based control、MPC、PPO/DQN 或 simple heuristic。CityLearn Challenge 也把常見評估目標定得很清楚，例如 peak demand、load shifting、整體成本等，這會讓你的

proposal 顯得很「像真的研究」。不過老實說，這題比庫存控制難一些，因為它的 state / reward / 控制目標通常更複雜，有些設定還帶 multi-agent 味道。 [citylearn.net+2](#)

3. 用 DRL 學習 Job-Shop Scheduling 的派工策略

這題是我給你的**第三順位**，也是三題裡研究味最重的一題。NeurIPS 2020 的 *Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning* 就是非常典型的範例：它不是手工設計 dispatching rule，而是讓 DRL 自動學習高品質的 priority dispatching rule，並且有官方實作 repo。後續也有多個 job-shop RL framework 與環境可參考。這表示它非常適合寫成「傳統 rule 很依賴專家知識，但 DRL 可以直接從狀態學 policy」的論文故事。 [NeurIPS 會議錄+5](#)

但我要先提醒你，這題雖然很漂亮，**也是三題裡最難的一題**。難點不只在演算法，而是在問題表示、排程狀態建模、reward 設計、以及和 heuristic / OR baseline 的比較。如果你現在想要的是「能穩穩做完 proposal，後續也有機會做出 prototype」，那它不如庫存控制穩；但如果你想讓題目看起來更學術、更像 conference paper 路線，這題很有競爭力。 [arXiv+2](#)

直接幫你做結論

如果你要的是**最穩、最容易成功、最不需要碰 agent 概念**，選：

庫存補貨策略最佳化

如果你要的是**最有真實世界應用感、最好講 motivation、最像智慧系統論文**，選：

建築能源 / 電池排程控制

如果你要的是**最有研究味、最像正式 DRL 論文題目**，選：

Job-Shop Scheduling 派工策略學習。 [arXiv+2](#)

以你現在的程度，我的單一建議是：

先選「庫存補貨策略最佳化」。它最符合「你還不想碰 agent、但又要做出像樣 DRL proposal」這個條件。你如果點頭，我下一則就直接幫你這題寫成教授格式的 **題目 + Abstract + Introduction + Related Work + Proposed Design + Experiment Plan**。

釐清一點：我當然要碰 agent, drl 就是 s(state), a(action), r(reward) 組件組成, agent 概念在這之中非常重要, 我前面說不要碰 agent 是指 "ai agent" 如 openclaw, antigravity, claude code 這些, 不是 drl 世界中的 agent, 所以不要再說 **"不需要碰 agent 概念"**, 請知悉

1. 以需求感知深度強化學習進行都市路口自適應交通號誌控制
2. DRL 做建築能源 / 電池排程控制

這兩個題目你覺得哪個適合套進教授 A to Z 的論文敘述流程 以及 兩者的難度差異
顯示更多顯示較少

好，以下這份就是我幫你整理的 **最佳 prompt**。

它的設計重點，是讓我下一回合直接沿著目前最成熟的交通號誌 DRL 工具鏈與 benchmark 脈絡工作：**SUMO-**

RL + Gymnasium + Stable-Baselines3 + RESCO，並且完整對齊你 ilearning 上的交付格式。SUMO-RL 官方明確支援以 SUMO 做 traffic signal control、相容 Gymnasium 與 Stable-Baselines3；RESCO 則是這個領域常見的 benchmark 脈絡。^{mf} Lucas Alegre+3

請你下一回合**直接整段貼上**即可：

你現在要正式開始執行我的「深度強化學習」期末專題 proposal 任務。

一、專案主題

我的 final project 題目已確定為：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Urban Intersection Traffic Signal Control

二、最重要的執行原則

1. 請先完整理解我先提供的 ilearning 作業說明、教授 A-to-Z 論文寫作影片、以及我上傳的所有參考檔案，再開始產出內容。
2. 你必須以「完成 ilearning 上要求的 Part A 內容」為核心目標。
3. 你要把自己當成：
 - 論文規劃顧問
 - 專案規格架構師
 - 簡報與口說稿設計師
 - GitHub 提交內容規劃者
4. 你必須明確區分：
 - ****DRL 的 agent**** (state / action / reward / policy / environment)
 - ****AI agent 工具鏈**** (如 Antigravity、Claude Code 等)
 本任務中可以提到前者，但不要把後者當成研究主題。
5. 你必須優先使用****官方文件、官方 repo、正式論文、benchmark 原始來源****作為主要依據。
6. 你必須主動上網搜尋最新且可信的資料，不可只靠記憶。
7. 除非真的必要，否則不要先問我澄清問題；請先做合理假設並明確寫出假設。
8. 全部回答都用****繁體中文****。
9. 若出現數學符號、RL 符號或較抽象名詞，請主動加上：
 - 符號注釋
 - 白話說明

三、ilearning 任務要求（以我提供的作業截圖為準）

請你依照 ilearning 上的要求處理，尤其注意：

Part A

需要有：

- github
- ppt
- video (錄製約 10 分鐘)

cover following section

1. Introduction / A to F
2. Related Work / NotebookLM
3. Proposed Design / 規格書

被劃掉的部分

- Detail Implementation
- Conclusion

這兩個被劃掉的部分，請****不要當作主要必交章節****，但若你判斷在 proposal 中需要少量提及，可用：

- Future Work
- Expected Implementation Plan

- Expected Outcomes

等較輕的方式處理，不要把它們寫成正式主章節。

Abstract 結構

請明確按照教授要求的這六段邏輯來寫：

1. Attention Getter / Motivation
2. But (挑戰)
3. Cure (我們提出什麼方法)
4. Development (方法設計基於什麼)
5. Experiments (怎麼驗證)
6. Findings (預期或初步可得的發現)

四、你必須先完成的工作順序

請嚴格依照以下順序輸出，不要跳步：

Step 1：先做「作業需求拆解」

請先用條列方式，精準整理：

- 教授真正要交的是什麼
- 哪些是必交
- 哪些不是這次 proposal 的主體
- 我這個題目最應該如何對齊這份作業

Step 2：做「題目定位與研究問題收斂」

請幫我把這個題目收斂成一個適合 proposal 的研究版本，包括：

- 研究背景
- 核心問題
- 研究動機
- 為何值得做
- 與傳統 fixed-time / actuated traffic signal control 的差異
- 我的研究主張 (novel angle)

請至少提出：

- 1 個主研究問題 (Main Research Question)
- 2~3 個子研究問題 (Sub-questions)

Step 3：建立「方法論框架」

請用 DRL 標準語言幫我定義：

- Environment (環境)
- Agent (智能體)
- State (狀態)
- Action (動作)
- Reward (回饋)
- Episode / Horizon
- Training objective

請特別說明：

- 什麼是 demand-aware
- demand-aware 要怎麼體現在 state、reward 或 control logic
- 為什麼這個設計是合理的

Step 4：幫我選擇最適合 proposal 的技術路線

請基於可信來源，幫我規劃：

- 模擬器 / 環境：優先考慮 SUMO / SUMO-RL
- RL 介面：Gymnasium
- 演算法：PPO、DQN 或其他更合理的 baseline / candidate
- 訓練框架：Stable-Baselines3 或其他合適方案
- benchmark / scenario 參考：如 RESCO

請你不要只列名字，要解釋：

- 為什麼這些工具最適合這個題目

- 它們彼此怎麼串起來
- 哪些是 proposal 階段該承諾的
- 哪些是 implementation 階段再處理即可

Step 5：完整寫出 proposal 內文初稿

請直接幫我產出可用於作業的**正式 proposal 初稿**，內容至少包含：

5.1 題目

- 中文題目
- 英文題目

5.2 Abstract

必須嚴格依照 A-to-Z 六段邏輯寫成一個完整 abstract

5.3 Introduction / A-to-F

請把 A-to-F 的邏輯完整展開成 proposal 的導論內容，不要只寫大綱

5.4 Related Work / NotebookLM

這一段要同時滿足兩件事：

1. 真正的 related work
2. 回應教授要求中的「NotebookLM」

請你幫我設計成最合理的寫法，例如：

- 我如何利用 NotebookLM 協助整理文獻
- 但真正 related work 仍要回到正式文獻與 benchmark
- 要避免讓 NotebookLM 看起來像研究主體本身

這一段請包含：

- 傳統交通號誌控制
- RL / DRL traffic signal control
- benchmark 與 simulation 生態
- 我题目的定位

5.5 Proposed Design / 規格書

這一段請用「規格書」思維來寫，包含：

- 系統流程
- 模組拆解
- 輸入 / 輸出
- 交通路口設定假設
- agent interaction flow
- reward design rationale
- baseline design
- evaluation metrics
- 預期實驗情境
- 可能的 ablation study

Step 6：把 proposal 內容轉成「教授會聽得懂、10 分鐘可講完」的簡報結構

請產出一份完整的 PPT 規劃，至少包含：

- 每一頁的頁標題
- 每頁要講的重點
- 建議圖表 / 視覺元素
- 每頁 speaker notes
- 哪些頁適合強調 Attention Getter / But / Cure / Development / Experiments / Findings
- 哪些頁最適合放方法架構圖、state-action-reward 圖、baseline 比較圖

Step 7：產出 10 分鐘口說稿

請幫我寫一份可直接錄影的逐頁口說稿，要求：

- 全長約 9~10 分鐘
- 中文口說自然
- 不要太像念論文
- 但要保有學術感
- 每頁稿都要清楚標註對應投影片頁碼
- 要有開場、轉場、收束

- 結尾不要硬寫 conclusion, 可用 :
 - proposal summary
 - expected impact
 - future validation

等方式處理

Step 8: 規劃 GitHub 交付內容

請幫我設計一個適合 proposal 階段的 GitHub repository 結構, 包括 :

- README.md
- docs/
- slides/
- proposal/
- refs/
- assets/
- script/
- environment.yml 或 requirements.txt (若合理)
- 其他你認為 proposal 階段應該放的内容

並請直接產出 :

1. repo 樹狀結構
2. README.md 初稿
3. 各資料夾應放内容
4. 建議檔名命名規則

Step 9: 交付清單總整理

最後請幫我整理成 :

- 我可以直接拿去交作業的項目
- 我需要自行補上的最少外部操作
 - 例如: 實際上傳 GitHub
 - 實際錄製影片
 - 實際上傳 YouTube
- 一份最終 submission checklist

五、研究內容要求

請務必讓研究内容具備以下特徵 :

1. 題目核心必須是 :
 - **需求感知深度強化學習**
 - **都市路口**
 - **自適應交通號誌控制**
2. 明確比較 :
 - fixed-time control
 - actuated control
 - demand-aware DRL approach
3. 盡量讓 proposal 呈現出 :
 - 研究動機強
 - 問題明確
 - baseline 合理
 - 指標具體
 - 系統規格清楚
 - 有 benchmark / toolchain 支撐
 - 適合後續交給 AI coding agent 執行
4. 請不要納入 CityLearn 的任何成分
5. 請不要偏向智慧建築、電池、EV、需求響應
6. 請不要改題目方向
7. 請不要把主體做成「AI agent 系統」
8. 主體必須始終是 DRL traffic signal control

六、資料與來源要求

請你必須同時整合以下來源脈絡：

我已提供給你的資料

- ilearning 作業截圖
- 教授 A-to-Z 影片連結
- media.mp4
- Frank_oral_defense_0725_v7.pptx
- Smart_City_Drone_Law_Enforcement.pdf
- 2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

你要自行補充查找的外部資料

請優先搜尋並引用：

- SUMO 官方文件
- SUMO-RL 官方文件 / 官方 repo
- Gymnasium 官方文件
- Stable-Baselines3 官方文件
- RESCO benchmark
- traffic signal control 相關正式論文
- 必要時可補充 OpenReview、arXiv、官方 GitHub repo

引用品質要求

- 優先用官方與 primary sources
- 不要用低品質部落格當主要依據
- 需要有 citation
- 若是較新的工具或文件，必須查最新資訊
- 不確定的地方要明講，不要硬猜

七、輸出格式要求

1. 請用清楚的 markdown 結構輸出
2. 每個大段落要有標題
3. 提供表格時請真的有比較價值，不要為了表格而表格
4. 有些內容要「可直接拿去交」
5. 有些內容要「可直接複製到 GitHub / PPT / 講稿」
6. 若你能直接產出檔案草稿，也請一併產出
7. 如果內容太長，請你分段清楚輸出，但不要偷省略重要內容

八、最終目標

你這次不是只回答我問題，而是要幫我完成：

- 作業要求解讀
- proposal 主體內容
- related work 方向
- proposed design 規格書
- ppt 章節規劃
- 10 分鐘口說稿
- GitHub 提交內容規劃
- 最終 checklist

請現在開始執行，並先從：

****Step 1：作業需求拆解****

開始。

等你下一回合把這段貼回來後，我就會正式開始執行。

貼上的 Markdown (1).md

檔案

以下先只做你要求的 **Step 1：作業需求拆解**。

我會先把「教授到底要你交什麼」拆乾淨，並把你的題目〈以需求感知深度強化學習進行都市路口自適應交通號誌控制〉對齊到這份作業。

Step 1：作業需求拆解

1. 教授真正要交的是什麼

你這次要交的，不是完整研究成品，也不是把系統全部做完；本質上是「一份可被說服、可被展示、可被後續實作的 **proposal package**」。

也就是說，教授要看的是你有沒有把下面這四件事講清楚：

1. 你要解什麼問題
2. 這個問題目前有什麼缺口
3. 你打算怎麼設計方法
4. 你未來打算怎麼驗證它

這和你提供的 A-to-Z 結構完全一致：

Attention Getter / Motivation → But → Cure → Development → Experiments → Findings。

所以這次作業的核心，不是 **implementation completeness**，而是 **research story completeness**。

2. 必交項目是什麼

依你提供的 ilearning 作業要求，**Part A 必交三個載體**：

- **GitHub**
- **PPT**
- **Video (約 10 分鐘)**

而且內容必須 **cover following sections**：

1. **Introduction / A to F**
2. **Related Work / NotebookLM**
3. **Proposed Design / 規格書**

換句話說，教授不是只要一份 proposal 文字，而是要你把 proposal 做成三種形式的可展示成果：

- GitHub：作為專案交付與結構化存放空間
- PPT：作為正式報告載體
- Video：作為 10 分鐘口頭簡報證據

所以這份作業的「最終交付物」不是單一文件，而是 **proposal 的完整展示套件**。

3. 哪些不是這次 proposal 的主體

ilarning 上被劃掉的兩部分是：

- **Detail Implementation**
- **Conclusion**

這代表教授已經明示：

這次不要求你進入實作細節，也不要求你做出完整研究結論。

因此這次作業裡，下面這些東西都不應該成為主體：

- 程式碼工程細節
- 訓練超參數表列到極細程度
- 模型已收斂、已 SOTA、已完整驗證的口氣
- 像 final paper 一樣的完整 conclusion

你可以提，但只能用較輕的形式，例如：

- Expected Implementation Plan
- Preliminary Experiment Plan
- Expected Outcomes
- Future Validation

也就是說，這次的正確姿勢不是：

「我已經把系統完整做完，所以來報告結果」

而是：

「我已經把研究問題、方法設計、比較基準、驗證方式規劃完整，足以進入後續實作」

4. 這份作業的評分重心，實際上會落在哪裡

從作業結構看，教授很可能會重看這幾件事：

A. 題目是否夠清楚

不是只講「我要做交通號誌控制」，而是要精準到：

- urban intersection
- adaptive traffic signal control
- demand-aware DRL
- baseline 為 fixed-time / actuated

B. 研究問題是否具體

不是泛泛地說「提升交通效率」，而是要明講：

- 想改善什麼指標
- 與什麼 baseline 比
- 需求感知體現在哪裡
- 為何不是一般 DRL，而是 demand-aware DRL

C. Proposed Design 是否像「規格書」

也就是方法章節不能只講概念，必須有：

- environment
- state
- action
- reward
- scenario
- metrics
- baseline
- experiment plan

D. 10 分鐘簡報是否有完整敘事

你不是在念論文，而是在說服教授：

- 問題值得做
- 工具鏈合理
- 方法設計成形
- 後續可執行

5. 你的題目，怎麼對齊這份作業最漂亮

你的題目其實非常適合這份作業格式，因為它天然就能被拆成教授想看的三大章：

5.1 Introduction / A-to-F 很好寫

這題的動機非常直觀：

都市路口壅塞、固定時制不夠彈性、傳統感應式控制仍有限、尖離峰車流變化大。

這些都能自然對應 A-to-Z 的前半段：

- Attention Getter：城市壅塞與等待成本
- Motivation：交通號誌控制的重要性
- But：fixed-time 與 actuated control 的限制
- Cure：需求感知 DRL
- Development：基於 SUMO / SUMO-RL / Gymnasium / SB3 的設計
- Experiments / Findings：和 baseline 比較 delay、queue、travel time

這條故事線很順。

而且 SUMO-RL 官方本來就是為 **Traffic Signal Control** 提供 RL 環境，並且相容 Gymnasium 與 Stable-Baselines3，這讓你的題目不只是概念上可行，而是工具鏈上也非常對齊。  [lucasalegre.github.io+2](https://github.com/lucasalegre/sumo-rl)

5.2 Related Work / NotebookLM 也很好對齊

這題的文獻脈絡成熟，至少可以分三層：

- 傳統交通號誌控制：fixed-time、actuated
- RL / DRL traffic signal control
- benchmark / simulation 生態：SUMO、SUMO-RL、RESCO

特別是 SUMO 官方已清楚說明 **actuated traffic lights** 是根據交通流或其缺乏而切換，這使你能把傳統方法講得很具體，而不是只空泛地說「舊方法不好」。RESCO 則提供 realistic traffic signal benchmark，讓你在 related work 裡可以談標準化比較，而不只是列 paper。  [sumo.dlr.de+1](https://sumo.dlr.de)

至於 **NotebookLM**，在這份作業裡最合理的定位不是研究主體，而是：

- 文獻整理輔助工具
- Related work 觀念彙整工具
- 幫你整理 benchmark、baseline、術語差異的閱讀助手

也就是說，這一段應該寫成：

我利用 NotebookLM 輔助整理與歸納文獻，但真正的研究依據仍以官方文件、正式論文與 benchmark 原始來源為主。

這樣最穩。

5.3 Proposed Design / 規格書 更是這題的強項

這題很容易寫成「規格書」而不是空話，因為核心模組很清楚：

- **Environment (環境)**：SUMO / SUMO-RL
- **Agent (智能體)**：DRL traffic signal controller
- **State (狀態)**：queue length、waiting time、current phase、approaching demand

- **Action (動作)** : phase selection / hold / switch
- **Reward (回饋)** : delay、queue、pressure 或其組合
- **Baselines** : fixed-time、actuated
- **Metrics** : average waiting time、queue length、travel time、throughput

SUMO-RL 官方文件明寫 state 和 reward 都容易修改，且可 single-agent / multi-agent；SUMO 官方文件也支援 actuated 與其他交通燈控制機制。這表示你的 proposed design 不只是能寫，還能寫得很像真正要進入實驗。
by [lucasalegre.github.io+2](https://github.com/lucasalegre)

6. 你這份作業最應避免的錯誤

這是我目前最重要的提醒。

錯誤 1：把 proposal 寫成 implementation report

不要過早陷入：

- 已完成多少 epochs
- code 細節
- 訓練時間
- 所有超參數表

這次主體不是這個。

錯誤 2：把題目寫太大

你現在的題目應該先聚焦在：

- 都市單一路口
- 需求感知 DRL
- 與 fixed-time / actuated 比較

不要一開始就擴成：

- 大型全城市路網
- 多智能體協同全部路口
- V2X / edge sensing / 智慧城市大平台

proposal 階段要的是清楚而可執行，不是大而空。

錯誤 3：把 NotebookLM 寫成研究方法本體

教授要求裡的「Related Work / NotebookLM」比較像是閱讀與整理流程的一部分。若你把它寫成主研究方法，會偏題。

錯誤 4：把 video 當成只是念投影片

這 10 分鐘其實是 proposal 的口頭版。
你要做的是說服，不是朗讀。

7. 你這個題目目前最正確的作業定位

如果我把它濃縮成一句話，你這份作業最應該被定義成：

一份以 SUMO-RL 為核心工具鏈、以 fixed-time 與 actuated 為 baseline、聚焦單一路口 demand-aware DRL traffic signal control 的研究 proposal 套件。

這個定位之所以好，是因為它同時滿足：

- 研究問題清楚
- baseline 清楚
- 工具鏈成熟
- 簡報容易講
- GitHub 容易整理
- 後續可直接交給 coding agent 落地

8. Step 1 的最終結論

你這次作業真正要交的是：

一套完整的 proposal package，內容主體是：

- Introduction / A-to-F
- Related Work / NotebookLM
- Proposed Design / 規格書

載體則是：

- GitHub
- PPT
- 10 分鐘影片

而你的題目

〈以需求感知深度強化學習進行都市路口自適應交通號誌控制〉

非常適合這份作業，因為它能自然對齊：

- 教授要的 A-to-Z 敘事
- 成熟的 benchmark 與工具鏈

- 清楚的 baseline 比較
- 規格書式的方法設計

特別是 SUMO-RL、SUMO 官方 traffic lights 文件、SB3 與 RESCO 這些來源，已經足夠支撐你把 proposal 寫成一個不只是題目好聽，而是真的可被執行的研究計畫。  [lucasalegre.github.io+3](https://github.com/lucasalegre)

下一步我會進入 **Step 2：題目定位與研究問題收斂**。

Step 2：題目定位與研究問題收斂

好，以下進入 **Step 2：題目定位與研究問題收斂**。

這一步的目標，不是先把方法細節寫滿，而是先把你的題目從「好聽的方向」收斂成「可被寫成 proposal、可被驗證、可被後續實作」的研究版本。

Step 2：題目定位與研究問題收斂

一、我先替這個題目下工作定義

你的題目目前最適合被收斂成：

在單一都市化路口的模擬環境中，設計一個具需求感知能力的深度強化學習交通號誌控制器，並與 **fixed-time** 與 **actuated control** 比較其在等待時間、排隊長度與旅行時間上的表現。

這樣收斂的原因很明確：

- **交通號誌控制本來就是 RL 很自然的決策問題**：交通狀態是輸入，號誌 phase 是動作，延遲與壅塞可作為回饋。RESCO 論文也直接把這件事寫成「根據當前交通狀態輸出下一個 signal phases」的控制問題。

 [datasets-benchmarks-proceedings-neurips.cc](https://github.com/datasets-benchmarks-proceedings-neurips)

- **題目要先聚焦在單一路口、單智能體**，proposal 才會乾淨；SUMO-RL 官方也明確支援 single-agent 與 multiagent，且 state 與 reward 易於客製。  [lucasalegre.github.io+1](https://github.com/lucasalegre)

二、合理假設

為了讓 proposal 可寫、可做、可驗證，我先明確列出這一版的合理假設：

1. **研究場景先以單一都市路口為主**，不直接擴成全區域多路口協同。
2. **環境以 SUMO / SUMO-RL 模擬為主**，不主張這次 proposal 直接做真實道路部署。SUMO-RL 官方就是為 SUMO 上的 traffic signal control RL 環境設計。  [lucasalegre.github.io+1](https://github.com/lucasalegre)
3. **baseline 至少包含 fixed-time 與 actuated control**。SUMO 官方文件明確支援固定時制與 actuated traffic lights，且 actuated 是根據 gap 或 delay 動態調整。  Eclipse SUMO

4. **demand-aware** 的主張，不是另做交通流預測大模型，而是讓 agent 的 state / reward / control logic 更明確感知「當前與短時需求壓力」。這樣研究邊界會比較清楚。
5. **proposal** 階段以研究問題、系統規格、實驗設計為主，不承諾完整 deployment。

三、研究背景

在都市交通中，號誌化路口會顯著影響通勤延遲。RESCO benchmark 論文指出，都市通勤時間中約有 **12% 到 55%** 來自號誌化路口造成的停止或接近延遲，因此更好的號誌控制有潛力降低通勤時間、擁塞、排放與燃油消耗。

 [datasets-benchmarks-](#)

[proceedings.neurips.cc](#)

傳統交通號誌控制長期依賴固定時制或感應式控制。SUMO 官方文件說明，預設交通燈可用固定週期運作；而 actuated 版本則讓綠燈時長在一定範圍內變動，依據 induction loop 偵測到的 gap，或依 vehicle delay 來決定。這代表傳統方法確實已具備某種「動態性」，但其調整邏輯仍屬預先定義規則，而不是從資料互動中學得策略。 [Eclipse SUMO](#)

近年的 DRL 交通號誌研究快速成長。2024 年的 survey 指出，固定時制在面對日益複雜的都市交通需求時已顯侷限，而 DRL 已成為重要研究方向；但同時，該 survey 也強調許多 TSC 研究仍主要在理想化模擬環境中驗證，距離真實部署還有差距。RESCO benchmark 也進一步指出，過去不少方法在不同感測假設與較真實的非 stylized intersection layouts 下並不夠 robust。 [科學直接+2](#)

四、核心問題

把問題濃縮成一句話：

如何讓交通號誌控制器不只是「被動依規則延長或縮短綠燈」，而是能夠根據路口當下的需求壓力主動學出更好的 phase 決策策略？

更具體地說，這個題目的核心不是單純「用 DRL 控號誌」，而是：

在固定時制與感應式控制已存在的前提下，

需求感知 (demand-aware) DRL 是否能更有效率地利用即時交通狀態，降低延遲與壅塞？

五、研究動機

這個題目的動機可以分成三層。

1. 實務動機

都市路口的交通需求高度波動，尖峰、離峰、方向性流量不均、短時突增等情況都很常見。固定時制不會即時回應這種波動；即便是 actuated control，通常也只是對「目前是否還有車」做局部延長，未必能對整體需求壓力做更全面的平衡。SUMO 官方文件中，actuated 的本質仍是基於 gap 或 delay 的規則式調整。 [Eclipse SUMO](#)

2. 研究動機

RL/DRL 的優勢在於，它能把號誌控制視為序列決策問題：

智能體根據當前狀態選動作，並從回饋中逐步調整策略。RESCO 論文和 2024 survey 都把 TSC 視為典型的 DRL

[datasets-benchmarks-](#)

應用場景，並且指出 state、action、reward 的設計是這類研究的核心。[proceedings.neurips.cc+1](#)

3. 你的 proposal 動機

你的 proposal 最值得強調的，不是「我也做一個 DRL 控號誌」，而是：

我想把 DRL 的感知重心，從一般流量表徵，進一步收斂到“需求壓力”本身。

也就是說，讓 agent 更直接感知：

- 哪個方向現在排隊更嚴重
- 哪個相位的等待正在累積
- 哪些 incoming lanes 的需求壓力即將變成 bottleneck

這樣的 demand-aware framing，比單純說「用 PPO / DQN 控紅綠燈」更有研究主張。

六、為何值得做

這題值得做，不只是因為它是熱門 DRL 題，而是因為它同時滿足三個層面。

1. 問題重要

[datasets-benchmarks-](#)

號誌化路口直接影響通勤延遲、擁塞與排放。這是典型高社會價值議題。[proceedings.neurips.cc](#)

2. 研究上有缺口

RESCO benchmark 的重要訊息是：先前很多 RL 交通號誌方法，在較真實的 layout 與感測假設下不夠穩健。

2024 survey 也指出，很多研究仍停留在理想化模擬。這使得你的 proposal 可以合理主張：

不是只追求更高分數，而是要讓需求表徵與控制策略更貼近實際路口的決策需要。[OpenReview+2](#)

3. 方法與驗證環境成熟

SUMO、SUMO-RL、Gym-compatible tooling 與 RESCO benchmark 已形成相對成熟的研究生態。這代表這題不只是概念有趣，而是有標準化工具鏈支撐 proposal 後續落地。SUMO-RL 官方與 Stable-Baselines3 專案頁都明確指出其與 Gym / RL library 的相容性。[lucasalegre.github.io+1](#)

七、與傳統 fixed-time / actuated traffic signal control 的差異

這一段是 proposal 裡一定要講清楚的。

1. Fixed-time control

固定時制的本質是：

相位順序固定、週期固定、綠燈分配預先設定。

優點是簡單、穩定、易部署；缺點是對即時需求變動不敏感。SUMO 官方文件預設就是 fixed cycle，這也正好能當成最基本 baseline。 

2. Actuated control

感應式控制的本质是：

根據偵測到的車流 gap 或 delay，延長或縮短特定相位的綠燈時間。

它已經比 fixed-time 更動態，但仍主要依靠人工設計的規則與觸發條件。SUMO 官方文件也說明，actuated 與 delay_based 仍是在既有 phase sequence 下變動時長，而不是由學習器主動學得整體控制政策。 

3. Demand-aware DRL

你要做的不是單純的 adaptive，而是：

讓 agent 在每個決策點，根據需求壓力表徵主動決定下一步號誌控制行為，以最大化長期回報。

白話說：

- fixed-time：先排好時刻表
- actuated：看到車就多給幾秒
- demand-aware DRL：看整體需求壓力，再學「現在應該偏向服務哪個方向、何時切換、怎樣長期更划算」

這就是你題目的本質差異。

八、研究主張（novel angle）

我建議你的 **novel angle** 不要寫得太大，而要寫得剛好能在 proposal 站得住。

建議主張

本研究主張：在單一路口自適應交通號誌控制中，若將交通需求壓力明確納入狀態表徵與回饋設計，可望使 DRL controller 相較於 fixed-time 與 actuated control，更有效地平衡不同來向車流需求，進而降低整體等待時間與排隊長度。

這個主張有三個好處：

1. 不誇大

你不是說自己要發明全新 RL 演算法，而是聚焦在「demand-aware 設計」。

2. 和現有文獻對得上

2024 survey 特別指出 TSC 研究的主流差異常落在 **state representation、action selection、reward design**。你把 novel angle 放在這裡，最合理。 

3. 容易轉成規格

下一步到 Step 3 時，你就能自然展開：

- demand-aware state

- demand-aware reward
- 可能的壓力平衡控制邏輯

九、主研究問題 (Main Research Question)

Main Research Question

在單一路口都市交通場景中，需求感知深度強化學習是否能比 **fixed-time** 與 **actuated traffic signal control** 更有效地降低整體延遲與壅塞？

白話版：

如果讓號誌控制器更清楚感知需求壓力，它能不能真的比傳統方法更會控紅綠燈？

十、子研究問題 (Sub-questions)

Sub-question 1

哪些交通需求資訊最能有效表徵路口控制決策所需的 **demand pressure**？

白話版：

是 **queue length** 比較重要？還是 **waiting time**？還是每個 **incoming lane** 的需求分佈？

這一題對應的是 **state representation**。

Sub-question 2

需求感知應該主要體現在 **state** 設計、**reward** 設計，還是兩者同時設計，才能帶來較穩定的控制效果？

白話版：

只把需求資訊塞進觀察值就夠了，還是 **reward** 也要明確懲罰需求失衡與排隊累積？

這一題對應的是 **state-reward coupling**。

Sub-question 3

在不同交通需求強度或不均衡流量條件下，**demand-aware DRL** 相對於 **fixed-time** 與 **actuated control** 的優勢是否一致？

白話版：

需求感知是不是只在尖峰才有用？還是離峰、不均衡流量、突發壓力情境下也有價值？

這一題對應的是 **robustness across demand scenarios**。

十一、我建議你最終採用的研究版本

如果要把題目正式收斂成 **proposal** 可直接使用的版本，我建議你用下面這個表述：

中文版

本研究聚焦於單一路口都市交通場景，提出一種需求感知深度強化學習交通號誌控制方法。相較於固定時制與感應式號誌控制，本研究將交通需求壓力納入狀態表徵與回饋設計，探討其是否能在不同交通需求條件下更有效降低平均等待時間、排隊長度與旅行時間。

英文版

This study focuses on adaptive traffic signal control at a single urban intersection and proposes a demand-aware deep reinforcement learning approach. Compared with fixed-time and actuated control, the proposed method explicitly incorporates traffic demand pressure into state representation and reward design, and evaluates whether it can more effectively reduce average waiting time, queue length, and travel time under varying traffic demand conditions.

十二、Step 2 的最終收斂結論

你這個題目，最好的研究定位不是：

「我想用 DRL 來控交通號誌」

而是：

「我想驗證：若把需求壓力更明確地放進 DRL 的感知與優化目標中，是否能讓單一路口自適應交通號誌控制比 fixed-time 與 actuated control 更有效。」

這樣收斂後，題目就同時具備：

- 明確背景
- 清楚基準
- 合理缺口
- 可操作的研究問題
- 可延伸到 Step 3 的方法論框架

下一步就可以很自然進到 **Step 3：建立方法論框架**。

Step 3：建立方法論框架

以下進入 **Step 3：建立方法論框架**。

這一步的目標，是把你的題目從「研究問題」正式落成為一個 **可寫進 proposal 規格書、可交給後續實作、可拿來設計實驗** 的 DRL 方法框架。

Step 3：建立方法論框架

一、這個方法框架的核心定義

我先用一句話定義你這個方法：

本研究將單一路口交通號誌控制建模為一個單智能體深度強化學習問題，利用 SUMO / SUMO-RL 作為互動環境，讓 agent 根據路口即時交通狀態與需求壓力表徵，週期性選擇下一個綠燈 phase，以最大化長期交通效率。

這個定義和 SUMO-RL 的官方定位一致：SUMO-RL 是用 SUMO 建立 **Traffic Signal Control** 的 RL 環境，支援 Gymnasium 相容介面，並可客製 state 與 reward；若設為 `single-agent=True`，它就會表現成標準的單智能體 Gymnasium 環境。  Lucas Alegre+3

二、DRL 方法論的六個核心元件

1. Environment（環境）

正式定義

環境是 agent 與之互動的交通模擬系統；在本研究中，環境為 **SUMO 模擬器上的單一路口號誌控制場景**，並以 **SUMO-RL 的 SumoEnvironment** 作為 RL 封裝介面。SUMO 官方文件明確指出，其交通燈模擬可用來「模擬任何可想像的交通號誌演算法以供研究」，而 SUMO-RL 則提供 Traffic Signal Control 的 RL 介面。  Eclipse SUMO+1

白話說明

你可以把環境想成「會回應控制器的虛擬路口」。

agent 每做一次決策，環境就往前模擬一段時間，再回傳新的交通狀態和回饋。

本研究建議設定

- 場景：單一路口、都市化四岔或二向主幹道交叉口
- 平台：SUMO + SUMO-RL
- 互動方式：透過 TraCI 與交通燈控制器互動；SUMO 官方明說可透過 TraCI 將外部程式碼接到執行中的模擬。  Eclipse SUMO

2. Agent（智能體）

正式定義

Agent 是學習控制政策的決策者；在本研究中，它是一個 **demand-aware DRL traffic signal controller**，其任務是依據當前交通狀態，輸出下一個交通號誌 phase 決策，以最大化長期報酬。

白話說明

agent 就是你的「紅綠燈決策者」。

它不是照死規則運作，而是透過和環境反覆互動，慢慢學會什麼情況該放哪一邊的車。

在本研究中的角色定位

- 單智能體控制單一路口
- 不處理全城市協同，不做多路口多智能體
- 不把 AI agent 工具鏈當研究主體

- 只聚焦 DRL 裡的 agent：也就是 **看狀態、選動作、收回饋、學策略**

3. State (狀態)

正式定義

狀態 s_t 是 agent 在時間 t 所觀測到的環境資訊。

SUMO-RL 的預設 observation 已提供一個標準向量：

$\text{obs} = [\text{phase_one_hot}, \text{min_green}, \text{lane}_1\text{-density}, \dots, \text{lane}_n\text{-density}, \text{lane}_1\text{-queue}, \dots, \text{lane}_n\text{-queue}]$

其中官方說明：

- **phase_one_hot**：目前啟用綠燈 phase 的 one-hot 表示
 - **min_green**：目前 phase 是否已滿足最短綠燈時間
 - **lane_i_density**：第 i 條進入車道的車流密度
 - **lane_i_queue**：第 i 條進入車道的排隊比例
- 而且 observation function 可自訂。 [GitHub+2](#)

符號注釋

- s_t ：時間 t 的狀態
- t ：第 t 個決策時刻
- i ：第 i 條車道
- n ：進入路口的車道總數

白話說明

狀態就是「agent 現在看到的路口樣子」：

現在是哪個綠燈、某些方向是不是已經綠很久了、各車道有多擠、哪裡排隊最長。

4. Action (動作)

正式定義

動作 a_t 是 agent 在時間 t 對環境採取的控制行為。

SUMO-RL 官方定義其 action space 為 **離散型**，且每隔 **delta_time** 秒，交通號誌 agent 可選擇「下一個綠燈 phase configuration」；若發生 phase change，下一相位前會先插入 **yellow_time** 秒的黃燈。 [GitHub+1](#)

白話說明

動作不是直接控制每盞燈亮幾秒，而是：

在這個決策點，選下一個要服務的綠燈相位。

本研究建議設定

- 動作空間：離散 phase 選擇
- 決策頻率：每 **delta_time** 秒決策一次
- 切換規則：若相位改變，必經黃燈過渡
- 研究定位：先不做連續控制，不做秒級綠燈時長回歸

5. Reward (回饋)

正式定義

回饋 r_t 是環境對 agent 當前動作優劣的即時評分。

SUMO-RL 的預設 reward 是「**累積車輛延遲的變化量**」；官方說明其本質是總 delay 相對前一時間步的變化，且可透過 `reward_fn` 自訂其他 reward。 [GitHub+1](#)

白話說明

reward 可以理解成：

你這一步的號誌控制，到底有沒有讓整體交通變得更順。

6. Training Objective (訓練目標)

正式定義

訓練目標是學得一個政策 $\pi(a | s)$ ，使 agent 在整個 episode 中的**折扣累積回報**最大化。其一般形式可寫為：

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

符號注釋

- $\pi(a | s)$ ：政策，表示在狀態 s 下選擇動作 a 的規則
- $J(\pi)$ ：政策 π 的目標值
- $\mathbb{E}$ ：期望值，表示取平均意義下的表現
- T ：一個 episode 的終點
- γ ：折扣因子， $0 < \gamma \leq 1$ ，用來控制「重視眼前」還是「重視長期」
- r_t ：時間 t 的即時回饋

白話說明

不是只求「這一步看起來好」，而是要學一個**長期最划算**的控燈策略。

有時候現在先忍一下，之後整體路口會更順；這就是 DRL 跟簡單規則控制不一樣的地方。

三、本研究建議的方法論實體化版本

下面這張表，是你 proposal 裡最適合直接放進去的「方法論框架總表」。

元件	本研究定義	白話說明
Environment	SUMO 中的單一路口交通模擬，透過 SUMO-RL 封裝為單智能體 Gymnasium 環境	虛擬路口
Agent	demand-aware DRL 號誌控制器	學習怎麼切燈的決策者
State s_t	phase、min_green、lane density、lane queue，再擴充 demand-aware 特徵	現在路口長什麼樣
Action a_t	每隔 <code>delta_time</code> 秒選一個下一綠燈 phase	下一步放哪一邊車

元件	本研究定義	白話說明
Reward r_t	以 delay / queue / demand imbalance 組成的自訂回饋	這次控燈有沒有讓交通更順
Episode / Horizon	一段完整模擬時窗，例如單次通勤尖峰模擬	一整場交通情境
Objective	最大化長期折扣累積回報	學會長期最好的控燈策略

四、什麼是 demand-aware

這是你題目最重要的詞，必須講得非常清楚。

正式定義

在本研究中，**demand-aware** 指的是：

agent 不只被動讀取一般交通狀態，而是**顯式地感知不同來向與不同車道的需求壓力（demand pressure）**，並把這種壓力納入決策依據。

白話說明

不是只看「有沒有車」，也不是只看「綠燈是不是該換了」，而是更進一步看：

- 哪個方向現在車特別多
- 哪些 lane 的排隊正在快速累積
- 哪個 phase 若繼續不服務，接下來會變成 bottleneck
- 路口是否出現需求不平衡

五、demand-aware 要怎麼體現在 state、reward、control logic

我建議你採用 **state + reward 雙重體現**，control logic 則維持 RL policy 自動學習，不要手工加太多規則。這樣最乾淨。

A. 體現在 State（狀態）

SUMO-RL 預設狀態已經含有：

- current phase
- min green
- lane density

- lane queue
而且官方明示 observation function 可以自訂。 [GitHub+1](#)

因此你可以在 proposal 中主張：
在預設 observation 基礎上，擴充更直接反映需求壓力的特徵。

建議加入的 demand-aware state 特徵

1. Approach-level aggregated demand

把同一方向所有 incoming lanes 的 density / queue 做聚合。

目的：不只看單一車道，而是看整個來向壓力。

2. Queue imbalance

計算不同方向之間的排隊失衡程度。

目的：讓 agent 知道是不是某一側已經被長時間忽略。

3. Short-term arrival pressure

在一個短時間窗內估計接近路口的需求強度。

目的：讓 agent 不只看已排隊車，也看「快要形成排隊」的需求。

4. Phase service gap

追蹤某個 phase 距離上次被服務已過多久。

目的：避免某方向一直得不到服務。

B. 體現在 Reward (回饋)

SUMO-RL 預設 reward 是 delay 變化量，並允許自訂 `reward_fn`。這讓你非常適合在 proposal 中提出 demand-aware reward。 [GitHub](#)

我建議的 reward 形式

你可以在 proposal 中寫成：

$$r_t = -(\alpha D_t + \beta Q_t + \gamma I_t + \eta S_t)$$

符號注釋

- r_t ：時間 t 的 reward
- D_t ：整體延遲成本 (delay cost)
- Q_t ：整體排隊成本 (queue cost)
- I_t ：需求失衡成本 (imbalance cost)
- S_t ：切換成本 (switching cost)
- $\alpha, \beta, \gamma, \eta$ ：權重係數，用來平衡不同目標的重要性

白話說明

這個 reward 的意思是：

我希望 agent 同時少延遲、少排隊、少忽略需求不平衡，也不要亂切燈。

為什麼這樣設計合理

- D_t 對應交通效率核心目標
- Q_t 讓 agent 對壅塞更敏感
- I_t 是 demand-aware 的關鍵，避免只服務當前最方便的方向
- S_t 防止 agent 過度頻繁切換 phase，保留交通燈控制的實務合理性

C. 體現在 Control Logic（控制邏輯）

這裡我不建議你手工再寫一堆 if-else 規則，因為那會削弱 DRL 的研究主體。

你最好的寫法是：

demand-aware control logic 並非額外設計一個規則式控制器，而是透過 demand-aware state 與 reward shaping，讓 policy 在學習過程中自行內化需求導向的決策偏好。

白話說明

你不是手工告訴 agent 「哪邊排隊多就一定先放哪邊」，

而是讓它在學習中自己理解：

如果某邊需求壓力一直累積，長期來看就會讓 reward 變差。

這樣比較像真正的 DRL 研究。

六、為什麼這個 demand-aware 設計是合理的

這裡我幫你把論證寫成 proposal 口吻。

1. 它建立在 SUMO-RL 已有的 MDP 表徵之上

SUMO-RL 官方預設 observation 已包含 phase、density、queue，且 observation/reward 可修改。也就是說，你不是另起爐灶，而是在既有交通號誌 RL 表徵上做更有目的性的擴充。^{mf Lucas Alegre+1}

2. 它符合交通號誌控制的決策本質

路口號誌控制的核心，不只是看「眼前誰有車」，而是要在多個方向的需求壓力之間做平衡。

因此，把 demand pressure 顯式放進 state 與 reward，理論上比單純依賴 phase + queue 的一般設計更貼近控制目的。這一點屬於研究推論，但它是建立在 SUMO-RL 可自訂狀態與 reward，以及 SUMO 官方對 fixed / actuated 機制的說明之上。^{🌿 Eclipse SUMO+2}

3. 它能清楚區分你和 baseline 的差異

- **fixed-time**：預先設定時序
- **actuated**：基於 gap 或 delay 的規則式延長

- **demand-aware DRL**：學習如何在多方向需求壓力之間做長期平衡
SUMO 官方文件明確指出 actuated 的本質是根據 gap 或 vehicle delay 進行 phase prolongation，且預設仍是固定 phase sequence、只在 timing 上變動。這剛好能凸顯你的研究主張。 

七、Episode / Horizon 怎麼定義最合適

正式定義

一個 **episode** 可定義為一段完整交通模擬時窗，例如：

- 單次早高峰
- 單次晚高峰
- 或固定長度的路口運行時段

建議設計

- 每個 episode：例如 3600 秒或 7200 秒模擬時間
- 每個 decision step：每 `delta_time` 秒做一次動作選擇
- 每次 phase 改變：固定插入 `yellow_time` 秒黃燈
SUMO-RL 官方已明確說明動作是每 `delta_time` 秒選一次下一相位，且切換前會先經過黃燈。 

白話說明

episode 就是「讓 agent 控一次完整的交通情境」。
不是只看一兩秒，而是看它整場控下來到底順不順。

八、Training Objective 的 proposal 寫法

你可以在 proposal 中用下面這段文字：

本研究將單一路口號誌控制建模為一個折扣回報最大化的序列決策問題。智能體需在每個決策時刻根據當前交通狀態選擇下一綠燈相位，並透過延遲、排隊與需求失衡等組成的回饋訊號，逐步學得一個能在長期上降低整體交通成本的控制政策。

這段既保留學術感，也不會太硬。

九、這一版方法論框架，適合直接放進 proposal 的最終版本

方法論框架（可直接複製到 proposal）

本研究將都市單一路口自適應交通號誌控制建模為單智能體深度強化學習問題。環境採用 SUMO 模擬器，並以 SUMO-RL 封裝為 Gymnasium 相容之單智能體交通號誌控制環境。智能體於每個決策時刻觀測路口交通狀態，

包括當前綠燈 phase、最短綠燈是否已滿足、各進入車道之密度與排隊程度，並進一步擴充需求感知特徵，如方向性需求聚合、排隊失衡與短時間到達壓力等資訊。智能體的動作為於固定控制週期內選擇下一綠燈相位配置；若相位切換則自動插入黃燈過渡。回饋設計則以整體延遲、排隊成本、需求失衡與相位切換成本組成，使智能體學習在不同交通需求條件下平衡多方向服務需求，最終最大化折扣累積回報。相較於 fixed-time 與 actuated control，本研究的方法強調將 demand pressure 顯式納入狀態表徵與回饋設計，以提升對都市交通需求波動之適應能力。 ^{inf} Lucas Alegre+4

十、Step 3 的最終結論

這一版方法論框架已經把你的題目正式落成為一個可執行的 DRL 問題：

- **Environment**：SUMO / SUMO-RL 單一路口
- **Agent**：demand-aware DRL 號誌控制器
- **State**：phase、min green、density、queue，加上需求壓力特徵
- **Action**：離散 phase 選擇
- **Reward**：延遲 + 排隊 + 失衡 + 切換成本
- **Episode / Horizon**：完整交通時窗
- **Objective**：最大化長期交通效率

而且最重要的是，**demand-aware** 不是口號，而是已被落在 **state** 與 **reward** 的具體設計上。

下一步最自然就是進入 **Step 4：幫你選擇最適合 proposal 的技術路線**。

網路吞吐量設計可以應用 DRL 技術嗎？

可以，而且很適合。

你這裡說的「網路吞吐量設計」，通常可拆成幾類：**壅塞控制（congestion control）**、**頻寬配置 / 排程、路由決策**，以及**應用層的自適應傳輸**。這些問題都能被寫成 DRL 的序列決策問題，所以在研究上早就有人做。像 Aurora 就把 **Internet congestion control** 直接建成 DRL 問題；Pensieve 則把 **自適應影音碼率（ABR）** 用 RL 來學習；ns3-gym 與 NetworkGym 也都是專門為「把網路問題包成 RL 環境」而生的工具。 [✖] arXiv+3

最典型的做法，是把它寫成這種 DRL 框架：

- **State（狀態）**：目前吞吐量、RTT、封包遺失率、queue length、buffer occupancy、估計 bottleneck bandwidth、cwnd 等。
- **Action（動作）**：調整 sending rate、congestion window、排程權重、route/path 選擇、甚至頻寬切片配置。
- **Reward（回饋）**：通常不是只看 throughput，而是

$$r = \text{throughput} - \alpha \cdot \text{delay} - \beta \cdot \text{loss}$$

也就是「高吞吐」加上「低延遲、低丟包、低抖動」的綜合效用。Aurora 的核心就是讓 agent 從網路觀測值中學 sending rate control policy，而不是寫死規則。✖ arXiv+2

如果你問的是「**有沒有真的拿 DRL 來提升吞吐量**」，答案也是有。

Aurora 是最經典的例子之一，研究目標就是讓 learned congestion control policy 在不同網路條件下有效利用容量。後續也有資料中心場景的 DRL 壅塞控制研究，強調在高性能資料中心網路中同時兼顧 throughput 與 latency；另外在網路切片 / eMBB 場景，也有人直接把「enhance throughput to fit SLA」當成 RL 任務。

✖ arXiv+2

如果你把範圍放寬到「**跟吞吐量密切相關的網路控制**」，那應用更多。

例如 Pensieve 不是直接調 TCP，而是學習影片串流的 bitrate adaptation policy，本質上是在變動網路頻寬下平衡 video quality、rebuffering 與 smoothness；這同樣是在做「如何把 available throughput 用得更好」。SIGCOMM 2022 的 Genet 也顯示，學習式方法能同時改進 adaptive bitrate streaming 與 congestion control。📄 ACM Digital Library+1

但這類題目有幾個現實難點，你要先知道。

第一，**reward 很難設**：只追 throughput，常會把 queue 撐爆，造成高延遲或 bufferbloat。第二，**泛化不容易**：在某些拓樸、流量型態、bottleneck 條件下學到的策略，換個環境可能就失效。第三，**公平性與穩定性**很重要：學習式壅塞控制不只要快，還要能和 Cubic、BBR 之類傳統協定共存，不然實用價值會很有限。近期對 learning-based congestion control 的系統性研究，就特別強調 fairness、efficiency、responsiveness 和 reproducibility 這些問題。✖ arXiv+2

若你是站在**題目規劃**角度看，這類主題很適合 DRL，因為它和交通號誌一樣，天然就是：

- 有連續互動的環境
- 有明確控制動作
- 有長期效益而非只看單步收益
- 可以設計 baseline 與 metrics

只是它比交通號誌更抽象，也更吃網路背景。常用的研究工具是 **ns-3 + Gym**，像 ns3-gym 就是把 ns-3 跟 RL 介面接起來；NetworkGym 也主打用於 networking systems 的 RL benchmark。✖ arXiv+2

所以一句話回答你：

可以，而且是很正統的 DRL 應用方向。

最典型的切入點就是 **壅塞控制 / 速率控制**；若再往外擴，還能做 **頻寬配置、排程、路由、自適應串流**。只是如果你把它當 final project 題目，它會比「自適應交通號誌控制」更抽象、reward 更難講、baseline 與實驗解讀也更容易陷入 networking 細節。✖ arXiv+2

你若要，我下一則可以直接幫你吧「**網路吞吐量 DRL 題目**」也像交通號誌那樣，拆成 **state / action / reward / baseline / metrics / proposal 題目名稱**。

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

檔案

Smart_City_Drone_Law_Enforcement.pdf

PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

1. A to Z 論文撰寫框架: 有 2 份
2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
2. 我的所有程式碼工作都會交給 antigravity 執行工作且要用到工具鏈 `**"openspec"**`
3. final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色

顯示更多顯示較少

我看完你補的作業頁面與這批參考資料後，結論很明確：

可行，而且我建議你做。

但前提是你不能把題目做成「很大、很酷、最後做不完」的版本；你要做成的是一個**先能提案、再能收斂、最後真的能做出來的網路 DRL 專題**。以你這次 Proposal 的要求來看，真正需要的是：**研究問題清楚、baseline 明確、規格可執行、GitHub/PPT/10 分鐘影片能完整說服老師**。你前面上傳的 oral defense 參考就是標準的 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**；《智慧城市指揮中心》那份則是典型的 **痛點 → 舊方法侷限 → 新系統架構 → 驗證 → 風險 → 導入路線圖**；2330 那份則示範了 **執行摘要 → 關鍵重點 → 路徑推演 → 完成項目 / 限制 / 品質檢核** 的一頁式管理報告；而你提供的 ChatGPT 收斂過程文件，則把 **A-F abstract、MDP formalization、state/action/reward、以及先批判 state representation 再提出 cure** 的流程講得非常清楚。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port... From 0 to 1 Deep RL with SMC MDP

用這些參考資料倒推，你這個題目最穩的版本，不是「網路吞吐量設計應用 DRL」這麼大，而是：

我建議你正式採用的題目

中文

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

英文

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

我之所以選這個版本，是因為它同時滿足三件事：

ns-3 官方已經有成熟的 TCP 模型與多種 congestion control baseline，例如 CUBIC、NewReno、BBR；ns3-gym 官方就是把 ns-3 包成 RL 研究環境；Aurora 也已經證明「用 DRL 做 congestion control」是正式研究路線，而不是冷門怪題；Pantheon 則提供 congestion-control benchmark 的比較思維。這代表你的題目不是從零發明，而是建立在成熟研究脈絡上。🌱[NSNAM+3](#)

更重要的是，這個版本和你畢論的 IPFS / 區塊鏈網路協定背景有**知識連續性**，但不會把 final project 一開始就綁死在 IPFS 實作上。你可以把 IPFS 放在 **Motivation / Future Work / Extension** 裡，說明你希望先掌握底層傳輸吞吐與擁塞控制，再未來延伸思考對 P2P / IPFS 類系統的啟發。這樣做，既保留研究主線，也把風險壓到最低。

我對你這題的最終判斷

可行嗎？

可行。

我會建議嗎？

會，而且在你想加深網路知識、又想讓成果真的做出來的前提下，我會建議。

但條件是什麼？

你必須接受這三條鐵律：

1. 這次 final project 不直接做 IPFS 系統本體。
2. 只做單一瓶頸鏈路，不碰多路徑、不碰真實 Internet、大型 P2P overlay。
3. 先做 baseline，再做 RL；先做 DQN MVP，再考慮 PPO / 進階版本。

如果你守住這三條，這題就不是不可控高風險題，而是**可做、可交、可講、可 demo** 的網路 DRL 題。

為什麼這條路是風險最低的

因為你現在最需要的不是「最前沿」，而是「最能完成」。

ns-3 的 TCP 模型本來就支援多種 congestion control 演算法，包含 CUBIC、NewReno、BBR 等，這讓 baseline 天然存在。ns3-gym 的 README 也直接說它是把 OpenAI Gym 與 ns-3 整合，用來鼓勵 networking

RL research，並且建議使用 Linux / Ubuntu 22+。而且近年的 NS-3 + DRL congestion control 工作，也已經顯示出這條路能在模擬器中做到「吞吐量接近 baseline、但 RTT 顯著更低」的結果。 NSNAM+2

這代表你的專題不用證明「世界上第一次有人這樣做」，你要證明的是：

我能把這個成熟方向，收斂成一個清楚的研究問題，並產出一個有規格、有 baseline、有實驗矩陣、有視覺化成果的專題。

這就已經非常像高分作品了。

你這次應該採用的「同款研究流程」

你說要「做成同樣的研究流程」，我幫你把你提供的三類參考資料，轉譯成你的專題流程。

1) A-to-Z 論文撰寫框架

你這一軌要直接套用 **A-F abstract 邏輯** 與 **MDP 問題建模**。你提供的 ChatGPT 收斂文件其實已經示範得非常清楚：先把 Attention / But / Cure / Development / Experiment / Finding 釘死，再把問題 formalize 成 state / action / reward / policy / transition。對你的題目來說，最自然的 But 就是：

傳統壅塞控制是固定 heuristic，雖強但對多樣網路條件的適應性有限；吞吐量最佳化不能只看 throughput，還要同時處理 RTT、loss 與 bufferbloat。

然後 Cure 就是：

用 DRL 把壅塞控制建模成 sequential decision problem，學一個以 throughput-delay-loss 折衷

 Proceedings of Machine Learning Research

為目標的 policy。

From 0 to 1 Deep RL with SMC MDP

2) 專題提案成果軌

這一軌我建議你直接借三份參考資料的優點。

Frank 口試簡報的骨架你直接拿來用：

Motivation → Related Works → Proposed Method → Experimental Design / Expected Results。

Frank_oral_defense_0725_v7

《智慧城市指揮中心》這份的優點，是它很會講「舊法為什麼不夠」與「新系統整體路線」。你應該學它的敘事節奏：

現實痛點 → 傳統方法限制 → 新架構 → 驗證圖表 → 風險管控 → 落地路線圖。

這對你的題目非常有用，因為你也需要把 TCP baseline、DRL controller、實驗風險、以及後續 deployment 分開講。

Smart_City_Drone_Law_Enforcement

2330 那份雖然不是研究簡報，但它的管理價值很高：

執行摘要、關鍵位階／路徑、完成項目／未完成項目／品質檢核。

你完全可以把這個框架轉成專題版本，做成附加 handout 或 GitHub README 裡的 executive summary。

2330_SMC_FVG_MTF_Report_A4_Port...

3) 教授的 ChatGPT 收斂過程軌

這份最值得學的不是內容，而是**收斂方法**：

- 先定 abstract
- 再打 But
- 再 formalize 成 MDP
- 再擬 state / action / reward
- 再擬 experiments
- 再擬 finding

對你的 final project，這會是最穩定的流程，不要一開始就衝著程式碼跑。

From 0 to 1 Deep RL with SMC MDP

你的角色怎麼定義最合理

你自己已經講得很清楚：

你不是親自寫 code 的人，你是整個專案的規劃者與指令下達者。

所以這個專題的角色分工應該定成：

- **你**：PI / PM / Research Architect / Spec Owner
- **Antigravity**：Implementation Agent / Experiment Runner / Refactoring Agent
- **OpenSpec**：Single Source of Truth for requirements, design, tasks, acceptance criteria

這樣的分工其實非常合理。OpenSpec 官方就把自己定位成「先把規格談清楚，再讓 AI coding assistant 動手」的輕量 spec-driven framework；官方頁面也直接寫了它是 open source、無需 API keys、無需 MCP，而 GitHub README 明確說每個 change 可以有自己的 proposal、specs、design、tasks，目的就是讓人和 AI 在 code 之前先對齊。  [OpenSpec+1](#)

我幫你設計的最低風險推進步驟

下面這條路，是我認為**最能實現完成**、而且最符合你角色設定的版本。

Phase 0：先凍結題目與範圍

這一步只做 3 件事：

1. 凍結題目
以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化
2. 凍結場景
單一 bottleneck、單 sender / receiver，最多雙流共享瓶頸作為加分題
3. 凍結指標
throughput、RTT、packet loss、utility

這一步不要碰 IPFS，不要碰 QUIC，不要碰多跳路由。
因為只要題目一大，你的 final 完成率就直線下降。

Phase 1：先做 proposal，不做實作

這一階段只產出文檔，不產出 code。

你要完成：

- 一版 A-F abstract
- 一版研究問題
- 一版 MDP formalization
- 一版 baseline / metrics / experiment matrix
- 一版風險清單

這就是 proposal 的主體。

Phase 2：用 OpenSpec 建立專案真相來源

這一步是你角色的核心。

我建議你的 OpenSpec change 至少拆成 4 個：

change-01-project-charter

定義題目、範圍、成功標準

change-02-baseline-benchmark

定義 ns-3 baseline：CUBIC、NewReno、BBR；以及 throughput / RTT / loss 的量測方式

change-03-opengym-env

定義 ns3-gym 的 observation、action、reward、episode、scenario

change-04-drl-mvp

定義 DQN MVP、訓練回合、實驗矩陣、輸出圖表與 accept criteria

OpenSpec 官方強調「每個 change 有自己的 proposal、specs、design、tasks」，這正好非常適合你把 Antigravity 的工作切成明確包。  [GitHub](#)

Phase 3：Baseline 先行

這一步是整條路的保命符。

Antigravity 第一個真正要執行的，不是 DRL，而是：

- 在 ns-3 跑 CUBIC
- 在 ns-3 跑 NewReno
- 在 ns-3 跑 BBR
- 固定 2-3 組網路情境
- 產出 throughput / RTT / loss 曲線圖

ns-3 官方文件已明示它原生支援這些 TCP 演算法，所以這一步理論上是工具鏈內最穩的部分。

只要 baseline 先跑通，你就已經有：

- proposal 的 Experiments 設計
- final 的 baseline chapter
- demo 的第一批圖表

Phase 4：接上 ns3-gym，但先不訓練

這一步的目標只有一個：

確保 environment 的 reset / step / observation / action / reward 全部打通。

ns3-gym 官方 README 已明確列出安裝依賴與 branch 使用方式，像是 ns-3.40、opengym 目錄命名、Linux 依賴等。這一步做成了，後面才有 DRL 可言。

Phase 5：做 DQN MVP

我強烈建議第一版只做 **DQN + 離散 action**。

理由很單純：

- DQN 原生適合離散 action
- 離散 action 比較容易定義驗收
- SB3 的 DQN 文件成熟
- 你的 goal 是先完成，不是先最優雅

動作可以設成：

- rate ↓
- rate =
- rate ↑

或：

- cwnd ↓
- cwnd =
- cwnd ↑

只要先證明：

- 比 random 好
- 在部分場景下接近某個 baseline
- 或在吞吐差不多時 RTT 更低

你的 final project 就已經成立。SB3 官方文件明寫 DQN 支援離散 action；PPO 可留作第二階段升級。

Phase 6：只有 DQN 成功後，才考慮 PPO / 擴展

這一階段是加分題，不是必做題。

可選擇的升級：

- PPO continuous action
- 共享瓶頸雙流公平性
- WebRTC bandwidth control 類擴展
OpenNetLab 的 gym 甚至直接提供了以 NS3 + WebRTC 做 bandwidth controller 的 gym。

但這些都要建立在 DQN MVP 已經跑通的前提下。

Phase 7：把成果封裝成高分交付物

這一階段你要做的不只是 report，而是像你給我的範例那樣，有完整展示層次。

我幫你定義的「100 分級」交付物

如果你要把這題做到最像高分作品，我建議你最後交出這 8 個東西：

1. 正式 proposal PDF / MD

內容結構：

- Abstract (A-F)
- Motivation
- Related Works
- Proposed Method
- Experimental Design
- Risk & Scope Control
- Expected Findings

2. 12-15 頁 PPT

骨架直接照 Frank oral defense 走，但把 Conclusion 改成：

- Expected Outcomes
- Risks
- Final Project Roadmap

3. 10 分鐘錄影講稿

節奏要像研究 proposal，不是念投影片。

4. GitHub repo

至少包含：

- README
- proposal/
- slides/
- docs/
- specs/ 或 openspec/
- experiments/
- figures/

5. OpenSpec 規格包

這是你和 Antigravity 的工作契約。

6. 一頁式 executive summary

學 2330 那種格式，做成：

- 執行摘要
- 研究假設
- baseline / DRL 路徑
- 已完成 / 未完成 / 風險

7. 圖表儀表板

至少有：

- throughput
- RTT
- loss
- utility
- baseline vs DQN

8. 最終 demo script

demo 不是展示 code，而是展示：

- baseline 圖
- RL 圖
- 指標對比
- 專案價值與未來延伸

你現在最不該做的 5 件事

1. 不要把核心問題綁到 IPFS 節點行為
2. 不要一開始就做多智能體
3. 不要一開始就做 QUIC / 真實部署
4. 不要直接從 PPO 開始
5. 不要先寫 code 再補 spec

這五件事，任何一件都足以讓專案失控。

我對你這題的最終建議

我的答案是：

做，而且值得做。

但要做的不是「很大的網路 DRL」，而是：

一個在 ns-3 裡可重現、可比較 baseline、可用 ns3-gym + SB3 訓練的單一瓶頸鏈路壅塞控制 DRL 專題。

這個版本最符合你現在的三個目標：

- 和畢論方向有知識連續性
- 能真正加深你對網路吞吐量 / 壅塞控制的理解
- 最有機會在 final project 裡**做完、講清楚、做漂亮**

下一步我建議你直接做什麼

下一步不要再討論大方向了，直接進入**規格凍結**。

我建議下一回合你要我做的是這個：

請直接幫我**把這個題目收斂成 OpenSpec 可用的專案規格包**，包含：

1. 題目正式版
2. A-F abstract
3. 研究問題
4. MDP (state/action/reward)
5. baseline / metrics
6. OpenSpec change 拆分
7. 給 Antigravity 的第一版主指令

這樣我們就會正式進入可執行階段。

下面這份 **最佳 prompt**，我已經把你現在最需要的「**Phase 0 一槌定音**」目標寫死了：它會強制先對齊你上傳的作業規格、口試型提案節奏、以及 A-F / MDP 收斂方式，再用官方 OpenSpec workflow、ns-3 TCP baseline、ns3-gym、Stable-Baselines3、Aurora 與 Pantheon 做深度搜證後，輸出一份**可直接凍結整學期方向**的規格定案。OpenSpec 本來就強調「先對齊 spec 再寫 code」，且每個 change 以 **proposal → specs → design → tasks** 組織；ns-3 官方已有 CUBIC、NewReno、BBR 等 TCP baseline；ns3-gym 明確是把 ns-3 與 RL 介面接起來；Aurora 則直接把 Internet congestion control 視為 DRL 問題。[GitHub+7](#)

另外，這份 prompt 也已內建你上傳材料的節奏要求：ilearning Part A 要交 GitHub / PPT / 約 10 分鐘影片、內容聚焦 Introduction / Related Work / Proposed Design；口試參考是 Motivation → Related Works → Proposed Method → Experimental Results；而你提供的收斂文件則強調 A-F abstract、MDP formalization、以及先批判 existing methods 再提出 cure 的敘事法。

貼上的 Markdown (1) Frank_oral_defense_0725_v7

From 0 to 1 Deep RL with SMC MDP

你現在要執行的是我 DRL Final Project 的 ****Phase 0：開路先鋒 / 一槌定音定案階段****。

你的唯一任務

你要用最嚴謹、最像研究計畫總設計師的方式，替我把整個期末專題方向 ****一次凍結****。

這不是 **brainstorming**，也不是泛泛建議，而是要做出：

- 題目正式定案
- 研究範圍正式定案
- **baseline** 正式定案
- 工具鏈正式定案
- OpenSpec 規格化推進框架正式定案
- 風險邊界正式定案
- Antigravity 後續實作指揮路線正式定案

我希望這次輸出具備「一槌定音」效果。

除非你找到足以推翻原方向的強力證據，否則你要以：

****「以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化」****

作為核心方向，替我完成 **Phase 0 定案**。

一、你必須先完整理解的前提

1. 我的角色

我不是親自寫 code 的人。

我在這個專案中的角色是：

- 題目規劃者
- 研究架構設計者
- 規格擁有者 (spec owner)
- 任務下令者
- 最終交付物整合者

2. 實作者角色

- 所有實作、程式碼、環境建置、benchmark 執行、重構、修正，都會交給 **Antigravity** 執行
- 專案規格化與實作交接必須使用 **OpenSpec**
- 你不可以把我當成要自己手寫全部程式的人
- 你必須為「AI coding agent + OpenSpec」這種工作方式設計 Phase 0 輸出

3. 我的真正目標

我不是要找最潮最炫的題目，而是要找：

> **最能完成、最能做出成果、最能先提案再收斂成期末專題、最有機會拿高分的網路吞吐量 + DRL 題目版本**

4. 我希望透過這個題目，順便累積什麼

- 深化電腦網路與壅塞控制知識
- 和我的畢業論文方向 (IPFS 區塊鏈網路協定) 建立知識連續性
- 但 **本次 final project 不必直接實作 IPFS**
- 必須以「能做完」優先，不要為了追求題目炫度而失控

二、你必須先閱讀與對齊的材料範圍

你必須先完整吸收、統整、比對下列三類材料，再開始做 Phase 0：

A. A to Z 論文撰寫框架 (2 份)

請抓出：

- 教授希望的 abstract 說服順序
- 如何從研究動機一路推到 proposed method / experiment
- 哪些內容適合放在 proposal，哪些不適合在 proposal 階段承諾過多

B. 專題提案成果 (3 份)

請抓出：

- 這類高分提案在版型、節奏、章節層次上怎麼鋪陳
- 如何從 Motivation → Related Works → Proposed Method → Experimental Results / Expected Outcomes
- 如何設計「像真的研究」的圖表、方法架構、路線圖、限制與未來工作
- 如何把成果做成 GitHub + PPT + 10 分鐘影片能講通的形式

C. 教授的 ChatGPT 收斂出專題提案對話過程 (1 份)

請抓出：

- A-F (Attention / But / Cure / Development / Experiments / Findings) 如何用在 proposal
- MDP / RL 題目應如何先收斂 abstract、再收斂 state/action/reward
- 怎麼用「先批判 existing methods 的弱點，再提出 cure」的方式建立研究價值
- 哪些收斂步驟適合被沿用到這次的網路吞吐量 DRL 題目

你必須把以上三類參考資料的優點全部轉譯到這次 Phase 0 中。

不要只做摘要，請把它們變成這次題目的規範。

三、Phase 0 的核心任務定義

Phase 0 的目標不是寫完整 proposal，而是做出 ****後續整學期專題都不需要再大改方向**** 的定案文件。

你要完成的是：

核心產出 1：題目凍結

請判斷並正式凍結以下內容：

- 最終中文題目
- 最終英文題目
- 一句話版本 (One-sentence thesis)
- 給老師看的學術版題目
- 給 GitHub / README / 簡報首頁使用的精簡版題目

核心產出 2：研究範圍凍結

請明確寫出：

- 這題 ****做什麼****
- 這題 ****不做什麼****
- 最低風險版本的研究邊界
- 哪些是 final project 必做
- 哪些是加分題
- 哪些絕對不能在這學期碰 (例如會讓專案爆掉的擴張)

核心產出 3：方法路徑凍結

請正式凍結：

- environment
- simulator
- RL interface
- baseline
- metrics
- reward philosophy
- 模型初始候選 (例如 DQN MVP 是否比 PPO 更適合作為 first milestone)
- final project 的最小可行成果 (MVP) 長相

核心產出 4：OpenSpec / Antigravity 推進路線凍結

請把這題正式轉成可交給 Antigravity 的 spec-driven 專案路線，至少定義：

- OpenSpec 的主 change 架構
- 每個 change 的目的
- 哪個 change 是 Phase 1 / Phase 2 / Phase 3 的先後順序
- 哪些文件應該先寫、哪些程式工作可以後做
- 如何避免 AI coding agent 因需求模糊而偏航

核心產出 5：風險最小化策略凍結

請用研究 PM 的角度，替我做：

- 風險清單
- 風險等級
- 觸發條件
- 避免方式
- 替代方案
- 什麼時候該停損、降階、縮範圍

四、你必須優先使用的資料來源

你必須先做完整、深度、嚴格的網路搜尋。

****禁止只靠記憶。****

優先級 1：官方 / primary sources

請優先搜尋並依賴：

- OpenSpec 官方網站與官方 GitHub README / docs
- ns-3 官方文件
- ns3-gym 官方 GitHub README / docs
- Stable-Baselines3 官方文件

- Aurora 原始論文
- Pantheon 官方文件 / paper
- 若需要補 baseline, 優先找 ns-3 官方 TCP model docs

優先級 2：正式論文 / benchmark

請找能支撐下列判斷的正式來源：

- DRL for congestion control 是否是正統研究方向
- 單一 bottleneck 是否是合理的收斂起點
- throughput 不應單獨作為唯一目標，必須與 delay / loss / fairness 權衡
- ns-3 / ns3-gym 是否適合做 final project 的最小可行環境
- DQN / PPO 在這題裡哪個更適合 Phase 1 MVP

優先級 3：必要時補充實作導向資料

若官方文件不足，再補：

- 高品質 GitHub repo
- 有明確社群採用脈絡的教學
- 但不能讓低品質部落格成為主要依據

五、你必須遵守的搜尋與判斷原則

1. 你要把自己當成「研究總設計師」，不是一般聊天助理
2. 你要主動比對多個來源，不可憑單一來源草率定案
3. 若有工具鏈過舊、已停止維護、或相依性風險高，必須明確點出
4. 若某條技術路徑理論上可行但實務上高風險，也必須說清楚
5. 你必須把「可完成性」放在最高優先，不要只談研究美感
6. 你必須假設：
 - 我可以理解網路基本概念
 - 我不是親手寫所有程式的人
 - 我需要的是「可以指揮 AI agent 實作的 spec-ready 設計」
7. 你不能模糊帶過任何會影響後續方向的大決策

六、這次 Phase 0 必須回答的核心問題

你必須明確回答以下問題，而且要有證據支撐：

1. 這個題目做成 final project **真的可行嗎？**
2. 在所有「網路吞吐量 + DRL」可能題型中，哪一個最適合：
 - 先提案
 - 再收斂
 - 最後做出來
3. 最低風險版本的題目應該是：
 - 單一瓶頸？
 - 多流共享瓶頸？
 - ABR / DASH？
 - QUIC？
 - 其他？
4. 為什麼你最終推薦的版本，比其他近似題型更適合這學期的 final project？
5. OpenSpec 應如何介入，才能讓後面 Antigravity 實作時不偏航？
6. 哪些內容應該直接被寫進 Phase 0 定案文件，成為後續不可輕易更動的「source of truth」？

七、這次輸出時，你的回答格式必須嚴格如下

請務必按以下章節輸出，不得跳章、不得省略：

Phase 0：定案總結

先用 150–250 字講清楚這次定案的最終結論。

1. 作業需求與參考資料的轉譯結論

請說明：

- 你如何把我提供的三類參考資料，轉譯成這次题目的 Phase 0 規格
- 哪些部分是必須被繼承的敘事方式
- 哪些部分是不能照搬、必須改寫成網路 DRL 專題的

2. 深度搜尋後的技術路線候選比較

請列出至少 4 條候選技術路線，例如：

- 單一瓶頸鏈路壅塞控制
- 多流共享瓶頸公平控制
- ABR / DASH 自適應串流
- QUIC / 其他

然後做比較分析，至少比較：

- 與「網路吞吐量 + DRL」的貼合度
- Proposal 適配度
- Final 可完成率
- 工具鏈成熟度
- baseline 清楚度
- 風險程度

最後給出明確排名。

3. 最終推薦題目（正式定案）

請正式給出：

- 中文題目
- 英文題目
- One-sentence thesis
- 題目定位一句話
- 為什麼它是第一名

4. 研究範圍凍結（Do / Not Do）

請用非常清楚的方式列出：

- 本專題要做什麼
- 本專題不做什麼
- 最低風險版本
- 加分擴展版本
- 明確禁止擴張的內容

5. 方法與工具鏈凍結

請正式凍結：

- 模擬器
- RL 介面
- baseline
- 初始演算法候選
- reward 設計哲學
- metrics
- MVP 標準
- 產出圖表種類

每一項都要說明「為什麼這樣定」。

6. OpenSpec 專案推進藍圖

請把這題轉成 OpenSpec-friendly 的規格藍圖，至少產出：

- repo / 規格層級架構建議
- OpenSpec changes 拆分
- 每個 change 的名稱、目的、輸出物
- 哪些 change 應先做
- 哪些 change 可後做
- 如何讓 Antigravity 依此執行而不偏航

7. 風險清單與降階策略

請列出至少：

- 主要風險
- 風險等級
- 為何是風險
- 預防方式
- 若發生時的降階替代方案

8. Phase 0 最終決策表

請整理成最終決策表，至少包含：

- 題目
- 場景
- baseline
- metrics
- toolchain
- MVP
- 不做事項
- 後續第一步

9. 給 Antigravity 的開工前總指令（草案）

請替我寫一段「開工前總指令」，目的是之後我可以直接拿來當 Antigravity / OpenSpec 的上游規格說明。這段要夠明確、夠嚴格、夠具體。

八、關鍵約束（非常重要）

1. 你不能只是「推薦題目」；你要做的是 ****Phase 0 定案****
2. 你要把「可完成率」放在第一優先
3. 你要明確納入：
 - 我是 spec owner，不是手寫 code 的人
 - Antigravity 是實作者
 - OpenSpec 是必用規格流程
4. 你要把最後輸出寫到足以作為：
 - 整學期方向凍結文件
 - 後續 proposal 的上游定義
 - 後續 OpenSpec change 的母規格
5. 你若認為原本我目前想法中的某條路徑不夠穩，必須直接糾正，不要迎合
6. 你必須誠實指出你不確定的地方
7. 你必須在結論中明確表態：
 - 我應不應該做這題
 - 如果做，應該做哪個版本
 - 為什麼

九、輸出語言與品質要求

- 全文用繁體中文
- 結構要非常清楚
- 不可偷省略細節
- 可直接拿來做下一步專案規格
- 必須像「研究總設計師 + 技術 PM + 論文規劃顧問」共同產出的結果
- 所有重要判斷都要有可信來源支撐
- 若有數學或 RL 符號，請補符號注釋與白話解釋

十、現在開始執行

請先做完完整深度搜尋與比對，再開始輸出。

這次不要先問我問題，除非你真的遇到無法合理推定的關鍵缺口。

請直接開始執行：

****Phase 0：定案總結****

把這段原封不動貼回來，我就會正式開始做 Phase 0。

Phase 0：定案總結

Phase 0 最終定案：你應該做這題，而且版本要正式凍結為「以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化」。

這是「網路吞吐量 + DRL」方向中，最適合先提案、再交給 Antigravity 以 OpenSpec 推進、最後產出可展示成果的版本。它不直接碰 IPFS、不做真實 Internet 部署、不做多路徑或 QUIC 協定實作，而是先在 **ns-3 單一 bottleneck link** 中建立可重現實驗，使用 **ns3-gym** 將網路模擬包成 RL 環境，再以 **Stable-Baselines3 DQN** 做最小可行 DRL agent，並與 **NewReno / CUBIC / BBR** 等 TCP baseline 比較 throughput、RTT、loss 與 utility。這條路保留你和 IPFS / 區塊鏈網路協定畢論的知識連續性，但避開 P2P overlay、真實節點、跨層協定整合的高風險泥沼。後續所有實作工作，必須先經 OpenSpec 拆成 proposal / specs / design / tasks，再交給 Antigravity 執行。

1. 作業需求與參考資料的轉譯結論

1.1 從 ilearning 作業要求轉譯出的 Phase 0 規格

這份 final project proposal 的核心不是先把系統完整做完，而是要交出一套能被老師快速理解、能被後續實作、能用 GitHub / PPT / 約 10 分鐘影片完整展示的 proposal package。你先前整理的作業規格明確要求 Part A 需要 **github**、**ppt**、**video**，內容要覆蓋 **Introduction / A to F**、**Related Work / NotebookLM**、**Proposed Design / 規格書**，並且被劃掉的 Detail Implementation / Conclusion 不應成為 proposal 主體。

貼上的 Markdown (1)

轉譯到本題後，Phase 0 必須先凍結三件事：

1. Introduction / A-to-F 的主敘事

從「網路應用越來越依賴穩定吞吐」切入，再指出傳統壅塞控制雖成熟，但面對多變網路條件時仍需在 throughput、delay、loss 之間做困難權衡。

2. Related Work / NotebookLM 的定位

NotebookLM 只能作為文獻整理工具，不能是研究方法本體。真正的 related work 要回到 Aurora、Pantheon、ns-3、ns3-gym、TCP baseline 與 RL-based congestion control。

3. Proposed Design / 規格書 的核心

要把研究設計寫成可執行規格：Environment、State、Action、Reward、Baseline、Metrics、Experiment Matrix、MVP。

1.2 從專題提案成果轉譯出的展示節奏

你上傳的 oral defense 範例採用非常標準的研究展示架構：**Motivation** → **Related Works** → **Proposed Method** → **Experimental Results** → **Conclusion**。其中 Motivation 先用現實問題與數據引出痛點，Related Works 再從舊方法一路鋪到新方法，Proposed Method 則把資料收集、訓練模型、偵測流程拆成模組，Experimental Results 用設備、資料比例、參數表、方法比較表來證明可信度。

Frank_oral_defense_0725_v7

轉譯到本題後，PPT / proposal 應該採用：

- **Motivation**：網路吞吐量與壅塞控制是分散式資料傳輸的底層問題
- **Related Works**：TCP NewReno / CUBIC / BBR → Aurora / RL-based CC → ns3-gym / benchmark
- **Proposed Method**：ns-3 bottleneck topology → OpenGym interface → DQN agent → metrics output
- **Expected Experimental Results**：throughput、RTT、loss、utility 比較圖
- **Limitations / Future Work**：未直接部署到 IPFS，但可作為 IPFS/P2P 傳輸優化的前置研究

1.3 從 A-F / MDP 收斂文件轉譯出的研究方法

你提供的 Deep RL + MDP 收斂文件最值得繼承的不是金融內容，而是**方法論順序**：先定義真正要訓練的不是「預測器」，而是「決策過程」；再把題目 formalize 成 State / Action / Reward / Transition / Policy；並且特別提醒 reward 設錯會讓 agent 學歪。

From 0 to 1 Deep RL with SMC MDP

轉譯到本題後：

- 不是訓練「吞吐量預測器」
- 而是訓練「壅塞控制決策器」
- State 不是隨便放 log，而是要放能反映網路狀態的觀測值
- Action 不能一開始做太大，要先離散化
- Reward 不能只看 throughput，必須同時懲罰 delay / loss

1.4 不能照搬、必須改寫的地方

不能照搬 oral defense 的「完整實驗結果」口吻，因為本階段是 proposal；所以你應該使用 **Expected Results / Evaluation Plan / Risk Control**，而不是宣稱已經完成所有訓練。不能照搬 SMC 文件中「PPO 是最佳」的結論，因為本題 first milestone 更適合離散動作的 DQN MVP；PPO 可保留為加分或後續版本。Stable-Baselines3 的 DQN 文件明確以 replay buffer、target network 等機制穩定神經網路 Q-learning，且其 policy 介面對應離散 action space；PPO 則較適合後續連續 action 或更一般的 policy optimization。

 [Stable-Baselines3 文檔+2](#)

2. 深度搜尋後的技術路線候選比較

2.1 候選路線總表

排名	技術路線	與「網路吞吐量 + DRL」貼合度	Proposal 適配度	Final 可完成率	工具鏈成熟度	Baseline 清楚度	風險程度	判斷
1	單一瓶頸鏈路壅塞控制	5	5	5	5	5	2	正式定案路線

排名	技術路線	與「網路吞吐量 + DRL」貼合度	Proposal 適配度	Final 可完成率	工具鏈成熟度	Baseline 清楚度	風險程度	判斷
2	多流共享瓶頸公平控制	5	5	3.5	4	4	3.5	加分擴展，不當 MVP
3	ABR / DASH 自適應串流	4	5	4	4	4	3	很好講，但偏應用層
4	QUIC 壅塞控制	5	4	2.5	3	3	4.5	題目亮，但協定工程風險高
5	AQM / Queue Disc DRL	4	3.5	3	4	3	4	系統味強，但較難向老師快速說清
6	IPFS / Bitswap / DHT DRL	4	3	1.5	2	2	5	與畢論貼，但本學期不碰

2.2 第一名：單一瓶頸鏈路壅塞控制

這條路最適合作為本學期 final project，因為它的研究主體乾淨：sender 經過 bottleneck link 傳輸到 receiver，agent 根據網路觀測調整傳輸行為，目標是在 throughput、RTT、loss 之間找到更好的折衷。ns-3 官方 TCP 文件說明其原生 TCP model 支援完整雙向 TCP、connection setup/close，並且支援多種壅塞控制演算法，包含預設 CUBIC、NewReno、BBR 等，這讓 baseline 非常穩。 [NSNAM](#)

ns3-gym 則明確把 ns-3 和 OpenAI Gym 整合，目的是鼓勵 reinforcement learning in networking research；這表示它正好適合本題要做的「網路模擬 + RL agent」橋接。 [GitHub](#)

Aurora 原始論文直接提出「deep RL-powered congestion control protocol」，並表示即使在有限訓練下也能

[Proceedings of
Machine Learning](#)

與 state-of-the-art competitive，這足以支撐本題是正式研究方向，而不是硬湊題。 [Research](#)

2.3 第二名：多流共享瓶頸公平控制

多流共享瓶頸更有研究味，因為它不只看 throughput，還會碰到 fairness。Pantheon 的論文與官方頁面都強調 congestion-control evaluation 需要共同 benchmark、共同介面與可重現比較；Pantheon 也支援 single flow、scripted cross traffic 等工作負載思維。 [Pantheon+1](#)

但這條路不適合當 Phase 0 MVP，因為 fairness 指標與多流互動會讓 state/action/reward 設計變複雜。它可以作為「第二階段加分題」，不該作為第一版必做。

2.4 第三名：ABR / DASH 自適應串流

Pensieve 是非常漂亮的 RL networking 題。它學習 adaptive bitrate algorithms，不用手寫固定控制規則，透過 reward signals 反映過去 bitrate 決策的 QoE。 [MIT CSAIL](#)

這題的優勢是簡報非常好講，影片 demo 也容易做；但它的問題是偏應用層 QoE，不是傳輸層壅塞控制。若你的目標是累積和 IPFS / 區塊鏈網路協定相關的底層網路知識，它不如單一瓶頸 CC 直接。

2.5 第四名：QUIC 壅塞控制

QUIC RFC 9002 是正式的 loss detection and congestion control 規範，因此題目有學術與協定正當性。

 RFC 編輯器+1

但這條路的風險在於：一旦真的做，容易從「DRL final project」變成「QUIC 協定實作 / 模擬器整合 / kernel or userspace transport engineering」。這不符合本學期最小風險策略。

2.6 第五名：AQM / Queue Disc DRL

這條路會把 agent 放在 router queue management，而不是 sender congestion control。它有研究價值，但 proposal 敘事要先解釋 AQM、bufferbloat、queue disc 與 congestion control 的分工。對 10 分鐘 proposal 來說，不如 sender-side bottleneck CC 直覺。

2.7 最終排名

正式排名：

1. 單一瓶頸鏈路壅塞控制與吞吐量最佳化
2. 多流共享瓶頸公平吞吐控制
3. ABR / DASH 自適應串流
4. QUIC 壅塞控制
5. AQM / Queue Disc DRL
6. IPFS / Bitswap / DHT DRL

Phase 0 定案：選第 1 題。

3. 最終推薦題目（正式定案）

3.1 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

3.2 英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

3.3 One-sentence thesis

本研究將單一瓶頸鏈路的壅塞控制建模為深度強化學習問題，透過 ns-3 / ns3-gym 建立可重現模擬環境，讓 agent 學習在吞吐量、延遲與丟包之間取得更佳平衡，並與 NewReno、CUBIC、BBR 等傳統 TCP baseline 比較。

3.4 題目定位一句話

這是一個以可重現網路模擬為基礎、聚焦傳輸層吞吐量與壅塞控制權衡的 DRL final project。

3.5 GitHub / README / 簡報首頁精簡版題目

DRL-Based Congestion Control over a Bottleneck Link

3.6 為什麼它是第一名

它是第一名，因為它同時滿足：

 Proceedings of
Machine Learning
Research

- **研究方向正統**：Aurora 已證明 DRL congestion control 是正式研究方向。  Research
- **工具鏈可行**：ns-3 有 TCP baseline，ns3-gym 可接 RL。  NSNAM+1
- **baseline 清楚**：NewReno / CUBIC / BBR 可直接成為比較對象。  NSNAM
- **成果可展示**：throughput / RTT / loss / utility 都能畫圖。
- **可與畢論連接但不綁死 IPFS**：它是 IPFS/P2P 傳輸效率研究的底層前置能力，但不把本學期拖進 IPFS 實作。

4. 研究範圍凍結 (Do / Not Do)

4.1 本專題要做什麼

本專題要做：

1. 在 **ns-3** 建立單一 bottleneck link 場景
2. 先跑傳統 TCP baseline：至少 NewReno / CUBIC / BBR
3. 使用 **ns3-gym** 將網路模擬轉成 RL environment
4. 定義最小可行 DRL task：State / Action / Reward / Episode
5. 使用 **Stable-Baselines3 DQN** 訓練第一版 agent
6. 比較 DRL agent 與 TCP baseline 的：

- throughput
- RTT
- packet loss
- utility

7. 產出 GitHub / PPT / 10 分鐘影片所需的圖表與規格文件

4.2 本專題不做什麼

本專題不做：

- 不直接改 Linux kernel TCP stack
- 不做真實 Internet 部署
- 不做 QUIC 實作
- 不做 IPFS 節點實作
- 不做 Bitswap / DHT / libp2p 行為修改
- 不做多路徑傳輸
- 不做大型 topology
- 不做 multi-agent congestion control
- 不承諾 outperform 所有 TCP baseline
- 不以 Pantheon 作為必裝依賴，只把它作為 benchmark 思想與 related work 參考

4.3 最低風險版本

MVP 版本：

- Topology：single sender → bottleneck link → receiver
- Baseline：NewReno / CUBIC / BBR
- RL：DQN with discrete action
- Action：increase / keep / decrease sending-rate 或 cwnd-like control signal
- Metrics：throughput、RTT、loss、utility
- 情境：2-3 組固定網路條件

4.4 加分擴展版本

只在 MVP 成功後才允許碰：

- 多流共享瓶頸
- cross traffic
- fairness metric

- PPO continuous action
- variable bandwidth trace
- IPFS / P2P motivation extension chapter

4.5 明確禁止擴張的內容

以下內容本學期禁止作為主體：

- IPFS Bitswap DRL
- IPFS DHT routing DRL
- QUIC congestion-control implementation
- true deployment
- kernel-level TCP
- multi-agent RL
- large topology
- Pantheon full integration as mandatory deliverable

5. 方法與工具鏈凍結

5.1 模擬器：ns-3

正式凍結：使用 ns-3。

理由：ns-3 是網路研究常用模擬器，其 TCP model 官方支援多種 congestion control baseline；這直接滿足本題需要可重現網路實驗與 baseline 比較的需求。  [NSNAM](#)

5.2 RL 介面：ns3-gym

正式凍結：使用 ns3-gym。

理由：ns3-gym 官方定位就是整合 OpenAI Gym 與 ns-3，以促進 networking research 中的 RL 使用。這正好是本題的關鍵橋樑：C++/ns-3 負責網路模擬，Python/RL agent 負責學習控制。  [GitHub+1](#)

5.3 Baseline：NewReno / CUBIC / BBR

正式凍結：至少 NewReno、CUBIC、BBR。

理由：ns-3 官方 TCP 文件明確列出這些 congestion control algorithms；CUBIC 也是 ns-3 TCP 的預設之一。

 [NSNAM](#)

Baseline 層級：

- 必做：NewReno、CUBIC

- 強烈建議：BBR
- 加分：Vegas / Westwood / DCTCP

5.4 初始演算法候選：DQN MVP，PPO 加分

正式凍結：第一版使用 DQN。

理由：為了降低 Antigravity 實作風險，Phase 1 需要離散 action。DQN 是最直覺的離散控制路線；PPO 雖穩定，但適合在 MVP 完成後再用於連續 action 或更複雜策略。Stable-Baselines3 的 DQN 文件說明其使用 replay buffer、target network 等機制穩定神經網路 Q-learning；PPO 則透過 clipping 避免 policy update 過大。^{Stable-Baselines3 文檔+1}

5.5 Reward 設計哲學

正式凍結：reward 不得只看 throughput。

建議 reward：

$$r_t = \alpha \cdot T_t - \beta \cdot RTT_t - \gamma \cdot L_t$$

符號注釋：

- r_t ：第 t 個時間步的 reward
- T_t ：throughput，吞吐量
- RTT_t ：round-trip time，往返延遲
- L_t ：loss rate，丟包率
- α, β, γ ：權重，用來決定重視吞吐、延遲或丟包的程度

白話說明：

這個 reward 的意思是「希望傳得快，但不能靠塞爆 queue 或大量丟包來換吞吐」。Aurora 和後續 congestion-control 研究都把 throughput、latency/loss trade-off 視為核心挑戰，而不是單純追求最大

^{Proceedings of Machine}

throughput。^{Learning Research+1}

5.6 Metrics

正式凍結以下 metrics：

- Average throughput
- Average RTT
- Packet loss rate
- Utility score
- Training reward curve
- Baseline comparison table

加分 metrics :

- Jain's fairness index
- Queue occupancy
- Convergence stability
- Generalization across bandwidth / delay settings

5.7 MVP 標準

MVP 完成標準 :

1. ns-3 baseline 能跑出 NewReno / CUBIC / BBR 圖表
2. ns3-gym environment 能 reset / step / return observation / action / reward
3. DQN agent 能完成訓練流程
4. 至少產出一張 baseline vs DRL comparison 圖
5. README 能重現執行流程
6. PPT 能用 10 分鐘講清楚研究價值

5.8 產出圖表種類

必做圖表 :

- Topology diagram
- MDP diagram : state / action / reward
- Baseline throughput comparison
- RTT comparison
- Loss comparison
- Reward curve
- Final utility comparison

加分圖表 :

- Cross-traffic scenario comparison
- Multi-flow fairness chart
- Ablation: reward without delay penalty vs with delay penalty

6. OpenSpec 專案推進藍圖

6.1 OpenSpec 為什麼必須介入

OpenSpec 官方說明，它的目的就是在寫 code 前，讓人類與 AI coding assistant 先對齊要做什麼；每個 change 有自己的 proposal、specs、design、tasks，避免需求只存在聊天紀錄而導致 AI 實作偏航。

 [GitHub+1](#)

所以 OpenSpec 在本專題中的角色是：

把你的研究決策轉成 Antigravity 可執行、可驗收、可追蹤的規格。

6.2 Repo / 規格層級架構建議

```
drl-congestion-control/
├── README.md
├── proposal/
│   ├── phase0_decision.md
│   ├── abstract_af.md
│   └── research_questions.md
├── docs/
│   ├── background_congestion_control.md
│   ├── related_work.md
│   ├── methodology_mdp.md
│   └── risk_register.md
├── openspec/
│   ├── project.md
│   └── changes/
│       ├── 01-project-charter/
│       ├── 02-ns3-baselines/
│       ├── 03-ns3-gym-env/
│       ├── 04-dqn-mvp/
│       └── 05-reporting-figures/
├── src/
│   ├── ns3/
│   ├── gym_env/
│   ├── agents/
│   └── analysis/
├── experiments/
│   ├── configs/
│   ├── logs/
│   └── results/
├── figures/
├── slides/
└── scripts/
```

6.3 OpenSpec changes 拆分

change 01：project-charter

目的：凍結題目、範圍、非目標、MVP。

輸出物：

- project charter
- do / not do list
- final project success criteria
- risk boundary

優先度：最高，第一個做。

change 02：ns3-baseline-benchmark

目的：先跑 TCP baseline，不碰 RL。

輸出物：

- ns-3 single bottleneck topology
- NewReno / CUBIC / BBR scripts
- throughput / RTT / loss logging
- baseline comparison figures

優先度：第二。

change 03 : ns3-gym-environment

目的：把 ns-3 場景包成 RL environment。

輸出物：

- observation space
- action space
- reward function
- reset / step interface
- random agent smoke test

優先度：第三。

change 04 : dqn-mvp-agent

目的：建立第一個可訓練 DRL agent。

輸出物：

- SB3 DQN training script
- evaluation script
- reward curve
- DQN vs baseline comparison

優先度：第四。

change 05 : reporting-figures-and-demo

目的：把成果轉成 final project 交付物。

輸出物：

- final figures
- PPT outline
- 10-min script
- GitHub README
- demo flow

優先度：第五。

6.4 哪些 change 應先做

必須順序：

1. 01-project-charter
2. 02-ns3-baseline-benchmark
3. 03-ns3-gym-environment
4. 04-dqn-mvp-agent
5. 05-reporting-figures-and-demo

禁止直接跳到 DQN。

沒有 baseline，就沒有可信比較。沒有 ns3-gym smoke test，就沒有 DRL 訓練。

6.5 如何避免 Antigravity 偏航

每個 change 都必須包含：

- Goal
- Non-goals
- Inputs
- Outputs
- Acceptance criteria
- Test command
- Expected artifacts
- What not to modify

Antigravity 每次只拿一個 change 做，不允許一次吃完整專案。

7. 風險清單與降階策略

風險	等級	為何是風險	預防方式	降階替代方案
ns3-gym 安裝卡住	高	C++/Python/版本依賴容易衝突	固定 Ubuntu + ns-3 版本；先跑官方 example	改用簡化 Python Gym toy bottleneck env
baseline 跑不出圖	高	沒 baseline 就無法比較	先做 change 02，不碰 RL	只保留 CUBIC / NewReno，BBR 改加分
RL reward 學歪	高	只追 throughput 會造成高 RTT / loss	reward 同時含 throughput、RTT、loss	先用 utility function 做評估，不宣稱優於所有 baseline
DQN 表現差	中	RL 不一定贏傳統 TCP	目標改成「分析 trade-off」	展示 DRL 在某些情境下

風險	等級	為何是風險	預防方式	降階替代方案
	高		而非「全面超越」	RTT/loss 較佳
action 設計太難	中高	cwnd / rate 控制若接不進 ns-3 會卡住	先用離散 action	改成外部 rate limiter / simplified control
PPO 太早導入	中	連續 action 會增加 debug 成本	Phase 1 僅 DQN	PPO 移到 Future Work
多流公平性爆炸	中	fairness / multi-flow state 複雜	MVP 單流	多流只做附錄或加分
IPFS 牽連過深	高	會變成 P2P 系統專案	IPFS 只放 motivation / future work	不進行任何 IPFS 實作
影片講不清楚	中	網路控制抽象	做 topology + MDP + metrics 三張核心圖	用「水管瓶頸」比喻說明
Antigravity 需求偏航	高	AI coding agent 容易自由發揮	OpenSpec change-by-change	每次只允許完成 acceptance criteria

停損規則

若 **change 02 baseline** 在兩天內跑不出至少一組可視化圖表，立即縮到 CUBIC vs NewReno。

若 **change 03 ns3-gym** 超過三天仍無法跑通 reset / step，立即啟動 simplified Gym bottleneck fallback。

若 **change 04 DQN** 無法穩定學習，final report 改成「DRL prototype + baseline trade-off analysis」，不硬宣稱 outperform。

8. Phase 0 最終決策表

決策項	Phase 0 定案
題目	以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化
英文題目	Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link
場景	Single sender → bottleneck link → receiver
不直接做	IPFS、QUIC、kernel TCP、多路徑、多智能體、真實部署
模擬器	ns-3
RL 介面	ns3-gym
初始 RL 演算法	DQN
加分演算法	PPO
Baseline	NewReno、CUBIC、BBR
Metrics	throughput、RTT、loss、utility、reward curve
Reward	throughput 正向，RTT / loss 懲罰
MVP	baseline 可跑 + ns3-gym 可 step + DQN 可訓練 + 圖表可輸出
OpenSpec 第一個 change	01-project-charter

決策項	Phase 0 定案
Antigravity 第一個實作任務	ns-3 baseline benchmark, 不是 DQN
後續第一步	產出 OpenSpec project-charter 與 change 01

9. 給 Antigravity 的開工前總指令（草案）

You are the implementation agent for a Deep Reinforcement Learning final project.

Project title:

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

The human user is the spec owner and research architect. Do not change the project direction without explicit approval.

Core goal:

Build a reproducible ns-3 based single-bottleneck congestion control experiment, then connect it to an RL workflow through ns3-gym and train a minimal DQN-based congestion-control agent.

Strict scope:

- Use ns-3 as the network simulator.
- Use ns3-gym as the RL interface.
- Use Stable-Baselines3 for the initial DQN agent.
- The first topology is single sender -> bottleneck link -> receiver.
- Baselines must include at least NewReno and CUBIC; BBR is strongly preferred if supported cleanly.
- Metrics must include throughput, RTT, packet loss, and a utility score.
- The first DRL action space must be discrete.
- Do not implement IPFS, QUIC, kernel TCP, multi-agent RL, multi-path routing, or real Internet deployment.

OpenSpec workflow:

Follow OpenSpec change-based development. Do not write implementation code before the corresponding OpenSpec change is approved.

Required OpenSpec changes:

- 01-project-charter
 - Define scope, non-goals, MVP, success criteria, and risk boundaries.
- 02-ns3-baseline-benchmark
 - Implement ns-3 single-bottleneck topology.
 - Run NewReno, CUBIC, and preferably BBR.
 - Export throughput, RTT, and loss logs.
 - Generate baseline figures.
- 03-ns3-gym-environment
 - Wrap the ns-3 scenario as an RL environment.
 - Define observation space, action space, reward function, reset, step, and episode.
 - Validate with a random agent smoke test.
- 04-dqn-mvp-agent
 - Train a Stable-Baselines3 DQN agent.
 - Use discrete actions such as decrease / keep / increase rate or cwnd-like control.
 - Compare DQN against baselines using the same metrics.
- 05-reporting-figures-and-demo
 - Produce final figures, README, experiment summary, and demo artifacts.

Acceptance criteria:

- Every change must have clear inputs, outputs, commands, expected artifacts, and validation steps.
- The project must be reproducible from README instructions.
- No scope expansion is allowed without spec owner approval.
- If ns3-gym integration fails, propose a fallback plan before changing direction.

Phase 0 最終表態

我建議你做這題。

而且不是「可以考慮」，是可以正式定案。

正式版本就是：

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

它不是你目前所有「網路吞吐量 + DRL」候選中，最能兼顧**研究正當性、可完成率、工具鏈成熟度、baseline 清楚度、與畢論方向延伸性**的路線。接下來不要再擴題，下一步應該直接產出 **OpenSpec change 01：project-charter**。

Phase 1：先做 proposal，不做實作

這一階段只產出文檔，不產出 code。

你要完成：

- 一版 A-F abstract
- 一版研究問題
- 一版 MDP formalization
- 一版 baseline / metrics / experiment matrix
- 一版風險清單

這就是 proposal 的主體。

接下來我們推進到 **“Phase 1：先做 proposal，不做實作”**

（一樣先不用進入執行環節，請先替我產生能讓你最佳化精準工作流程的**最佳 prompt**，我才會再餵回去給你真正開始執行任務：「Phase 1：先做 proposal」）
顯示更多顯示較少

我先把 Phase 1 的工作邊界釘死：這階段**只產出 proposal 文件，不做任何程式碼與實驗**。這份最佳 prompt 已把 OpenSpec 的「先對齊規格再寫 code」精神、ns-3 / ns3-gym 的網路 RL 工具鏈、SB3 的 DQN / PPO 定位、以及 Aurora / Pantheon 這類壅塞控制研究脈絡都納入成明確工作規範。OpenSpec 官方強調人與 AI 在寫 code 前先對齊規格，且每個 change 可包含 proposal、specs、design、tasks；ns-3 官方 TCP model 支援多種 congestion control baseline；ns3-gym 是把 ns-3 與 Gym 整合給 networking RL 研究使用；SB3 文件也顯示 DQN 適合離散 action，而 PPO 可作為後續 policy optimization 路線。 [GitHub+4](#)

下一回合請直接把以下整段貼回來，我就會正式開始執行 **Phase 1：先做 proposal，不做實作**。

你現在要執行的是我 DRL Final Project 的 **“Phase 1：先做 proposal，不做實作”**。

最高任務目標

請基於 Phase 0 已定案的方向，正式產出本期末專題 proposal 的核心文件包。

Phase 0 已正式凍結題目為：

****中文題目****

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

****英文題目****

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

Phase 1 的任務不是實作、不是寫 code、不是開始訓練模型，而是產出一組足以支撐後續 GitHub / PPT / 10 分鐘影片 / OpenSpec / Antigravity 實作的 proposal 主體文件。

你要讓這份 Phase 1 輸出具備以下效果：

1. 老師看得懂這題為什麼值得做
2. 研究問題明確
3. MDP / DRL formulation 成立
4. baseline、metrics、experiment matrix 清楚
5. 不過度承諾實驗結果
6. 後續能無縫轉入 OpenSpec change 01 / change 02
7. Antigravity 後續可以依照這份文件展開實作，不會偏航

一、我的角色與實作分工

1. 我的角色

我是：

- 題目規劃者
- 研究架構設計者
- 規格擁有者 (spec owner)
- 任務下令者
- 最終交付物整合者

我不是親自手寫全部程式的人。

2. Antigravity 的角色

Antigravity 是後續實作者，負責：

- 環境建置
- ns-3 baseline 實作
- ns3-gym environment 串接
- DQN MVP 實作
- 實驗執行
- 圖表產生
- bug fixing
- repo 整理

3. OpenSpec 的角色

OpenSpec 是本專案的規格中樞。

Phase 1 的輸出必須能轉成 OpenSpec change 的上游材料，尤其是：

- 01-project-charter
- 02-ns3-baseline-benchmark
- 03-ns3-gym-environment
- 04-dqn-mvp-agent

- 05-reporting-figures-and-demo

請在 Phase 1 結尾明確標註：哪些內容應該進入哪個 OpenSpec change。

二、Phase 1 嚴格邊界

1. Phase 1 要做什麼

Phase 1 只產出文件，必須完成：

1. 一版 A-F abstract
2. 一版研究問題
3. 一版 MDP formalization
4. 一版 baseline / metrics / experiment matrix
5. 一版風險清單

此外，為了讓 proposal 更完整，請你補充：

6. 題目定位與研究貢獻
7. Related Work 定位，不需長文獻回顧，但要能支撐 proposal
8. OpenSpec 對應表
9. 10 分鐘簡報可用的 proposal skeleton
10. Phase 1 最終交付清單

2. Phase 1 不做什麼

請嚴格禁止：

- 不寫實作 code
- 不假設已經跑出實驗結果
- 不宣稱 DRL 一定贏過 TCP baseline
- 不做 IPFS 實作
- 不做 QUIC 實作
- 不做 kernel-level TCP
- 不做 multi-agent
- 不做 multi-path routing
- 不做大型 topology
- 不把 Pantheon 當必裝工具
- 不把 PPO 當第一階段必做演算法
- 不把 phase 1 寫成 final report

Phase 1 的語氣應該是：

- proposed
- planned
- expected
- evaluated by
- we aim to
- this study formulates

而不是：

- achieved
- outperformed
- demonstrated
- results show

除非你明確寫成「預期結果」或「假設」。

三、你必須先做的資料搜尋與依據

請先做完整網路搜尋，不可只靠記憶。

優先搜尋來源

請優先使用官方與 primary sources：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns3-gym 官方 GitHub / ns-3 app 頁面
4. Stable-Baselines3 DQN 文件
5. Stable-Baselines3 PPO 文件
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充 RFC / 正式論文

你必須用搜尋結果確認的事

請確認並引用來源支撐：

1. ns-3 是否適合做 TCP / congestion control baseline
2. ns-3 是否支援 NewReno、CUBIC、BBR 等 baseline
3. ns3-gym 是否適合把 ns-3 包成 RL environment
4. DQN 是否適合離散 action 的 first MVP
5. PPO 為什麼適合留作後續擴充，而不是 Phase 1 的第一承諾
6. Aurora 為什麼能支撐「DRL for congestion control」是正統研究方向
7. Pantheon 為什麼適合放在 Related Work / benchmark 思想，而不是本專題必裝依賴
8. 為什麼 throughput 不能是唯一 reward / metric，必須同時考慮 RTT / loss / utility

四、必須對齊的作業與參考資料

請先對齊我先前提提供的作業與參考資料，不要只靠外部資料。

A. ilearning 作業規格

本 proposal 最終會服務於：

- GitHub
- PPT
- 約 10 分鐘影片

作業要求的主要內容是：

1. Introduction / A to F
2. Related Work / NotebookLM
3. Proposed Design / 規格書

被劃掉的：

- Detail Implementation
- Conclusion

因此 Phase 1 的內容要偏 proposal，而非 implementation report。

B. A-to-Z 論文撰寫框架

請承接教授要求的 abstract 六段邏輯：

1. Attention Getter / Motivation
2. But
3. Cure
4. Development
5. Experiments
6. Findings

請把這六段邏輯套用到本題，而不是泛泛寫摘要。

C. 專題提案成果參考

請繼承高分提案的節奏：

- Motivation
- Related Works
- Proposed Method
- Expected Experimental Results
- Risk / Limitation
- Roadmap

但請注意：本階段尚未實作，所以 Experimental Results 應寫成 Experiment Plan / Expected Findings。

D. 教授 ChatGPT 收斂提案的對話過程

請承接該文件中最重要的流程：

1. 先定義研究真正要解的決策問題
2. 再批判 existing methods 的弱點
3. 再提出 Cure
4. 再 formalize 成 MDP
5. 再定義 State / Action / Reward
6. 再設計 baseline 與 experiment

五、Phase 1 必須輸出的內容格式

請你嚴格按照以下章節輸出，不要跳章，不要偷省略。

Phase 1: Proposal 主體提案摘要

請先用 150–250 字說明：

- Phase 1 的目的
- 本題現在的研究定位
- 本 proposal 的核心主張
- 為什麼這份文件可以作為後續 OpenSpec / Antigravity 的上游基礎

1. 正式題目與定位

請輸出：

1.1 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

1.2 英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

1.3 簡報首頁版題目

請給 1 個較短、適合 PPT 首頁的英文題目。

1.4 GitHub repo 名稱建議

請給 3 個 repo name 候選，並推薦其中一個。

1.5 One-sentence thesis

請用一句話講清楚這份研究到底在做什麼。

1.6 研究定位

請說明這題是：

- networking systems 題
- DRL sequential decision-making 題
- congestion control / throughput optimization 題
- 不是 IPFS 實作題
- 不是 QUIC 實作題
- 不是真實 Internet deployment 題

2. A-F Abstract

請嚴格按照以下六段產出。

每一段都要有：

- English version
- 中文正式版
- 白話說明版

2.1 Attention / Motivation

要講出：

- 網路吞吐量對現代分散式系統、內容傳輸、區塊鏈 / P2P 類網路的重要性
- 壅塞控制是影響 throughput、delay、loss 的核心機制
- 與我的 IPFS / 區塊鏈網路協定畢論方向有知識連續性，但本專題不直接實作 IPFS

2.2 But

要批判 existing methods，但要精準，不要亂罵。

請重點放在：

- 傳統 TCP congestion control 依賴 rule-based / heuristic control logic
- 不同網路條件下 throughput、RTT、loss 權衡困難
- 只追 throughput 會導致 delay / loss / bufferbloat 問題
- 因此需要能根據觀測狀態調整策略的 sequential decision framework

2.3 Cure

要提出：

- 用 DRL 將 congestion control 建模為 sequential decision-making problem
- agent 根據網路狀態調整傳輸控制行為
- 目標不是盲目最大化 throughput，而是在 throughput、delay、loss 之間取得更佳平衡

2.4 Development

要說明方法基礎：

- ns-3 作為模擬器
- ns3-gym 作為 RL interface
- Stable-Baselines3 作為初始訓練框架
- DQN 作為 Phase 1 / MVP 的離散 action 候選
- PPO 留作 future extension

2.5 Experiments

要說明 planned experiments：

- baseline：NewReno、CUBIC、BBR

- scenario : single bottleneck link
- metrics : throughput, RTT, loss, utility
- experiment matrix : 不同 bandwidth / delay / loss / queue 或 cross-traffic 條件

2.6 Findings

注意：本階段不能宣稱已經有結果。

請寫成：

- Expected findings
- Hypothesis
- Evaluation goal

例如：

- 預期 DRL agent 未必全面超越所有 baseline
- 但可用來分析在特定情境下的 throughput-delay-loss trade-off
- 本研究的價值在於建立可重現 DRL congestion control prototype 與評估流程

3. 研究問題

請輸出：

3.1 Main Research Question

請給 1 個主研究問題。

3.2 Sub-questions

請給 3-5 個子研究問題，至少包含：

- state representation
- action design
- reward design
- baseline comparison
- generalization / robustness

3.3 Research Hypotheses

請給 2-3 個研究假設。

注意：假設不能寫成已證明的結果。

3.4 Research Contribution

請列出本專題可能貢獻，但要保守準確。

例如：

- 將 single-bottleneck congestion control 形式化為 DRL task
- 建立可重現 ns-3 / ns3-gym prototype 規格
- 比較 DRL 與 TCP baseline 的 throughput-delay-loss trade-off
- 為未來 IPFS / P2P 類網路協定研究提供底層知識基礎

4. MDP Formalization

請正式把本題建模為 MDP。

你必須包含：

4.1 MDP 總定義

請用正式符號寫：

$M = (S, A, P, R, \gamma)$

並補上：

- 符號注釋
- 白話說明

4.2 Environment

請定義：

- ns-3 single bottleneck link
- sender / receiver
- bottleneck capacity
- propagation delay
- queue / buffer
- optional cross traffic

4.3 Agent

請定義：

- DRL congestion-control agent
- agent 控制的是傳輸行為，不是整個網路
- Phase 1 先以 sender-side control abstraction 為主

4.4 State / Observation

請設計 MVP state, 至少包含：

- recent throughput
- RTT / delay signal
- packet loss signal
- queue / congestion indicator
- previous action

請同時提供：

- minimal state version
- enhanced state version
- 為什麼先用 minimal state 比較安全

4.5 Action

請設計 MVP action, 強烈建議離散 action, 例如：

- decrease sending rate / cwnd-like control
- keep
- increase sending rate / cwnd-like control

請說明：

- 為什麼第一階段不用 continuous action
- 為什麼 DQN 對此較適合
- PPO 何時才應該導入

4.6 Reward

請設計 reward, 不能只看 throughput。

請至少提出一個基本形式：

$$r_t = \alpha \cdot \text{throughput}_t - \beta \cdot \text{RTT}_t - \gamma \cdot \text{loss}_t$$

請提供：

- 符號注釋
- 白話說明

- 為什麼這樣能避免 agent 只追 throughput
- 可能的 reward normalization 問題
- 可能的 ablation: with / without delay penalty

4.7 Transition

請說明 transition 由 ns-3 network dynamics 決定, agent 不直接控制封包傳播, 只影響傳輸控制行為。

4.8 Policy / Objective

請寫出：

$\pi(a|s)$

以及最大化折扣累積回報的目標。

請加上符號注釋與白話說明。

5. Baseline / Metrics / Experiment Matrix

5.1 Baseline

請列出：

- NewReno
- CUBIC
- BBR

並說明為什麼選這三個。

請補充：

- 必做 baseline
- 強烈建議 baseline
- 加分 baseline

5.2 Metrics

請列出並定義：

- average throughput
- average RTT
- packet loss rate
- utility
- training reward curve
- convergence behavior

請說明每個 metric 在本題中的意義。

5.3 Experiment Matrix

請設計一個 proposal 階段可用的實驗矩陣。

至少包含：

- Scenario ID
- bandwidth
- propagation delay
- queue setting
- traffic pattern
- baseline methods
- expected observation
- risk level

請至少設計 4 組情境：

1. Stable low-delay bottleneck
2. Stable high-delay bottleneck

3. Variable bandwidth bottleneck

4. Bottleneck with cross traffic

5.4 Evaluation Philosophy

請明確寫出：

- 本專題不以「DRL 全面打敗所有 TCP baseline」作為唯一成功標準
- 成功標準是建立可重現環境、完成 baseline comparison、展示 DRL trade-off 分析能力
- 若 DRL 在特定場景低於 BBR / CUBIC，也可以成為合理 findings，只要分析完整

6. Related Work 定位

請不要寫成超長文獻回顧，但要為 proposal 建立可信定位。

請至少包含四類：

6.1 傳統 TCP congestion control

說明 NewReno、CUBIC、BBR 在本題裡的 baseline 角色。

6.2 Learning-based congestion control

說明 Aurora 為什麼是本題的重要 related work。

6.3 Network simulation and RL environment

說明 ns-3 / ns3-gym 的角色。

6.4 Benchmark and reproducibility

說明 Pantheon 的 benchmark 思想，並說明為什麼本專題不把 Pantheon 當第一階段必裝依賴。

7. 風險清單

請建立 Phase 1 proposal 版 risk register。

至少包含：

- Risk ID
- Risk description
- Probability
- Impact
- Prevention
- Fallback
- 是否影響 proposal
- 是否影響 final implementation

必須包含以下風險：

1. ns3-gym 安裝 / 版本問題
2. ns-3 baseline logging 不完整
3. DQN 不收敛
4. Reward 設計導致高吞吐但高 RTT / loss
5. BBR 在 ns-3 中跑法或版本相依性問題
6. Antigravity 實作偏航
7. 題目被誤解成 IPFS 實作
8. 實驗結果未超越 baseline

請為每項給 fallback。

8. OpenSpec 對應表

請把 Phase 1 產出的每一部分，對應到 OpenSpec changes。

表格欄位請包含：

- Phase 1 artifact
- 對應 OpenSpec change
- 用途
- 是否 required before coding
- 是否可後補

例如：

- A-F abstract → 01-project-charter
- MDP formulation → 03-ns3-gym-environment
- baseline / metrics → 02-ns3-baseline-benchmark
- risk register → 01-project-charter
- experiment matrix → 02 / 04

9. Proposal Skeleton

請產出一版 proposal 文件骨架，供我後續轉成正式報告。

結構至少包含：

1. Title
2. Abstract
3. Introduction / Motivation
4. Problem Statement
5. Related Work
6. Proposed Design
7. MDP Formulation
8. Experimental Plan
9. Risk and Scope Control
10. Expected Contributions
11. Roadmap

每一節請寫：

- Section purpose
- 要放什麼
- 不要放什麼
- 建議圖表

10. 10 分鐘簡報骨架

請產出 10 分鐘 proposal video / PPT 的頁面安排。

至少 10–12 頁，每頁包含：

- Slide title
- Key message
- Suggested visual
- Speaker note direction
- 對應 A–F 或 proposal section

注意：

這不是寫完整講稿，而是 Phase 1 的簡報骨架。

11. Phase 1 最終交付清單

請最後整理：

11.1 本次 Phase 1 已完成的文件項目

11.2 下一階段 Phase 2 前必須凍結的事項

11.3 哪些內容可直接轉入 OpenSpec

11.4 哪些內容可直接轉入 PPT

11.5 哪些內容可直接轉入 GitHub README

六、品質要求

1. 語言

- 全文用繁體中文
- 英文題目 / abstract 請提供英文版本
- 重要專有名詞保留英文並附中文解釋

2. 深度

請不要只列大綱，要寫到我可以直接拿去整理成 proposal 初稿。

3. 嚴謹度

所有重要判斷都要有可信來源支持。

若某件事是推論，請明確標示「這是基於來源的推論」。

4. 數學與符號

若出現：

- MDP
- S, A, P, R, γ
- $\pi(a|s)$
- r_t
- α, β, γ

請一定要附：

- 符號注釋
- 白話說明

5. 不准做的事

- 不要寫任何程式碼
- 不要產生完整 OpenSpec 檔案
- 不要開始實作 planning beyond proposal
- 不要宣稱已經有實驗成果
- 不要過度承諾模型表現
- 不要把本題改回自適應交通號誌
- 不要把本題改成 IPFS 實作
- 不要把本題改成 QUIC 實作

七、現在開始執行

請先搜尋官方文件與正式論文，再開始輸出。

請直接從：

Phase 1: Proposal 主體定案摘要

開始。

Phase 1：Proposal 主體定案摘要

本階段正式產出本專題的 **proposal 主體文件包**，不進入實作、不撰寫程式、不宣稱已有實驗成果。依照作業要求，本 proposal 服務於後續 **GitHub**、**PPT**、約 **10 分鐘影片**，並聚焦於 **Introduction / A-to-F**、**Related Work / NotebookLM**、**Proposed Design / 規格書**；被劃掉的 Detail Implementation 與 Conclusion 僅能以 Expected Plan、Risk、Future Work 輕量呈現。

貼上的 Markdown (1)

本題定位為一個 **networking systems + DRL sequential decision-making** 專題：將單一瓶頸鏈路壅塞控制建模為 MDP，使用 ns-3 / ns3-gym 作為後續模擬與 RL 介面設計基礎，並以 NewReno、CUBIC、BBR 作為 baseline。Phase 1 的核心主張是：本研究不追求「DRL 必然打敗所有 TCP」，而是建立可重現的 DRL congestion control proposal，分析 throughput、RTT、loss 之間的 trade-off。ns-3 官方 TCP 文件支援多種 TCP congestion control baseline，ns3-gym 則是為 ns-3 與 RL 研究整合而設計，足以支撐本 proposal 的技術路線。  nsnam.org

1. 正式題目與定位

1.1 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

1.2 英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

1.3 簡報首頁版題目

DRL-Based Congestion Control over a Bottleneck Link

這個版本短、清楚，適合放在 PPT 首頁與 GitHub README title。

1.4 GitHub repo 名稱建議

候選 repo name	評估
drl-bottleneck-congestion-control	最清楚，直接點出 DRL、bottleneck、congestion control
drl-throughput-optimization	簡潔，但太泛，沒有明確指出 congestion control
ns3-drl-congestion-control	工具鏈明確，但若後續 fallback 不用 ns3-gym，名字會稍微受限

推薦：drl-bottleneck-congestion-control

1.5 One-sentence thesis

This study formulates congestion control over a single bottleneck link as a deep reinforcement learning problem, aiming to learn a sender-side control policy that balances throughput, RTT, and packet loss under simulated network conditions.

中文白話版：

本研究把單一瓶頸鏈路的壅塞控制轉成 DRL 決策問題，讓 agent 根據網路狀態調整傳輸行為，目標不是只把吞吐量衝高，而是在吞吐量、延遲與丟包之間取得更合理的平衡。

1.6 研究定位

本題是：

- **networking systems 題**：核心是網路壅塞控制與傳輸效能。
- **DRL sequential decision-making 題**：核心是 agent 根據 state 選 action，再由 reward 更新 policy。
- **congestion control / throughput optimization 題**：核心指標是 throughput、RTT、loss、utility。
- **不是 IPFS 實作題**：IPFS 只作為長期研究動機與未來延伸背景。
- **不是 QUIC 實作題**：本階段不碰 QUIC 協定工程。
- **不是真實 Internet deployment 題**：本階段只在可控模擬環境中規劃。

這個定位承接你提供的 MDP 收斂文件精神：真正要做的不是預測器，而是「decision process under uncertainty」；對本題而言，就是壅塞狀態下的傳輸控制決策。

From 0 to 1 Deep RL with SMC MDP

2. A-F Abstract

2.1 Attention / Motivation

English version

Modern distributed systems, content delivery platforms, and blockchain or peer-to-peer networks rely heavily on efficient data transmission. In such systems, network throughput is not only a performance indicator but also a fundamental factor that affects responsiveness, scalability, and user experience. Congestion control plays a central role in determining how network flows balance throughput, delay, and packet loss. Although this project is motivated by the broader context of IPFS and blockchain-based network protocols, it focuses on the more fundamental problem of congestion control over a single bottleneck link.

中文正式版

現代分散式系統、內容傳輸平台，以及區塊鏈與 P2P 類網路，都高度依賴穩定且有效率的資料傳輸。在這些系統中，網路吞吐量不只是效能指標，更會直接影響系統回應速度、可擴展性與使用者體驗。壅塞控制是決定網路流量如何在吞吐量、延遲與丟包之間取得平衡的核心機制。本專題雖受到 IPFS 與區塊鏈網路協定研究方向啟發，但本研究將範圍收斂於更基礎且可控的單一瓶頸鏈路壅塞控制問題。

白話說明版

我不是一開始就做 IPFS，而是先做 IPFS 背後一定會碰到的底層問題：資料傳輸遇到瓶頸時，怎麼讓傳輸速度、延遲、丟包取得平衡。

2.2 But

English version

However, traditional TCP congestion control algorithms are largely based on rule-based or heuristic control logic. While these mechanisms are mature and widely deployed, they still face challenges in adapting to diverse network conditions where throughput, RTT, and packet loss must be jointly considered. Optimizing throughput alone may lead to excessive queueing delay, packet loss, or bufferbloat. Therefore, congestion control can be naturally reconsidered as a sequential decision-making problem in which the controller adapts its behavior according to observed network states.

中文正式版

然而，傳統 TCP 壅塞控制演算法多半建立在規則式或啟發式控制邏輯之上。這些方法雖然成熟且廣泛使用，但在面對多樣化網路條件時，仍必須同時處理吞吐量、RTT 與丟包率之間的權衡。若僅以最大化吞吐量為目標，可能導致過高的佇列延遲、封包遺失或 bufferbloat 問題。因此，壅塞控制可被重新建模為一個序列決策問題，使控制器能根據觀測到的網路狀態動態調整傳輸策略。

白話說明版

傳統 TCP 很強，但它通常是固定規則。網路狀況一變，單純追求傳得快可能反而讓延遲暴增或丟包變多，所以這題適合交給會觀察狀態並調整策略的 DRL agent。

2.3 Cure

English version

To address this challenge, this study proposes to formulate congestion control as a deep reinforcement learning problem. A DRL agent observes network conditions such as recent throughput, RTT, packet loss, and congestion indicators, and then selects a transmission control action. Instead of blindly maximizing throughput, the proposed framework aims to learn a policy that balances throughput improvement with delay and loss reduction.

中文正式版

為解決上述挑戰，本研究提出將壅塞控制建模為深度強化學習問題。DRL agent 會觀測近期吞吐量、RTT、丟包訊號與壅塞指標等網路狀態，並選擇相對應的傳輸控制動作。本研究的目標不是盲目最大化吞吐量，而是學習一個能在提升吞吐量、降低延遲與控制丟包之間取得平衡的控制政策。

白話說明版

我們讓 agent 看目前網路狀況，再決定要加快、維持或放慢傳輸。重點不是衝最高 throughput，而是找到整體比較划算的傳輸策略。

2.4 Development

English version

The proposed study is planned around a reproducible simulation-based workflow. The network environment will be modeled using ns-3, while ns3-gym will provide the reinforcement learning interface between the ns-3 simulation and a Python-based DRL agent. Stable-Baselines3 will be used as the initial training framework. For the MVP stage, DQN is selected as the first candidate because the planned action space is discrete, while PPO is reserved as a possible future extension for more flexible or continuous control settings.

中文正式版

本研究規劃採用可重現的模擬式研究流程。網路環境將以 ns-3 建立，並透過 ns3-gym 作為 ns-3 模擬器與 Python DRL agent 之間的強化學習介面。初始訓練框架規劃採用 Stable-Baselines3。MVP 階段優先選擇 DQN，原因是本研究初始 action space 設計為離散動作；PPO 則保留為後續擴充，用於更彈性或連續控制設定。

白話說明版

先用 ns-3 做網路模擬，用 ns3-gym 接 RL，再用 SB3 訓練 agent。第一版先用 DQN，因為動作設計成「減少 / 維持 / 增加」比較安全；PPO 先不承諾，放未來延伸。

ns-3 官方支援多種 TCP congestion control baseline；ns3-gym 官方明確說明其目標是整合 OpenAI Gym 與 ns-3 以促進 networking RL research；SB3 DQN 文件也標示 DQN 對應 Discrete action space，這支撐本研究把 DQN 作為 MVP 候選。  nsnam.org+2

2.5 Experiments

English version

The planned experiments will evaluate the proposed DRL-based congestion-control formulation under a single bottleneck link scenario. Traditional TCP congestion-control algorithms, including NewReno, CUBIC, and BBR, will serve as baselines. The evaluation will compare average throughput, average RTT, packet loss rate, and a combined utility score under different network conditions, such as stable low-delay bottlenecks, stable high-delay bottlenecks, variable bandwidth, and bottlenecks with cross traffic.

中文正式版

本研究規劃在單一瓶頸鏈路場景下評估所提出的 DRL 壅塞控制建模方式。傳統 TCP 壅塞控制演算法 NewReno、CUBIC 與 BBR 將作為比較 baseline。實驗將比較平均吞吐量、平均 RTT、封包遺失率與綜合 utility 分數，並規劃涵蓋穩定低延遲瓶頸、穩定高延遲瓶頸、變動頻寬瓶頸，以及含 cross traffic 的瓶頸場景。

白話說明版

我們會讓 DRL agent 和傳統 TCP 方法在同樣的模擬網路中比賽，但比的不是單一分數，而是 throughput、RTT、loss 這幾個指標一起看。

2.6 Findings

English version

As this is a proposal-stage study, no experimental results are claimed at this stage. The expected finding is not that the DRL agent will universally outperform all TCP baselines, but that the proposed framework can provide a reproducible environment for analyzing throughput-delay-loss trade-offs. The main value of this study is to establish a clear DRL formulation, baseline comparison plan, and evaluation workflow for congestion control over a bottleneck link.

中文正式版

由於本研究目前仍處於 proposal 階段，因此本階段不宣稱已有實驗結果。本研究的預期發現並非 DRL agent 會全面超越所有 TCP baseline，而是希望建立一個可重現的環境，用於分析 throughput、delay 與 loss 之間的權衡。本專題的主要價值在於建立清楚的 DRL formulation、baseline comparison plan 與壅塞控制評估流程。

白話說明版

我們不會先說 DRL 一定贏。真正的高分點是：把題目定義清楚、baseline 定義清楚、實驗矩陣定義清楚，最後能分析 DRL 在什麼情境下有價值、什麼情境下輸給傳統 TCP。

3. 研究問題

3.1 Main Research Question

How can congestion control over a single bottleneck link be formulated as a deep reinforcement learning problem to balance throughput, RTT, and packet loss compared with traditional TCP baselines?

中文正式版：

如何將單一瓶頸鏈路壅塞控制形式化為深度強化學習問題，並與傳統 TCP baseline 比較其在吞吐量、RTT 與丟包率之間的權衡能力？

3.2 Sub-questions

SQ1 : State representation

哪些網路觀測值最適合用來描述壅塞控制 agent 的 state ?

候選包含 recent throughput、RTT、loss、queue / congestion indicator、previous action。

SQ2 : Action design

在 MVP 階段，離散 action 是否足以表達有效的傳輸控制行為？

例如 decrease / keep / increase sending rate 或 cwnd-like control。

SQ3 : Reward design

如何設計 reward，使 agent 不會只追求 throughput，而能同時考慮 delay 與 loss？

SQ4 : Baseline comparison

DRL agent 在單一瓶頸鏈路中，和 NewReno、CUBIC、BBR 相比，能呈現出哪些 throughput-delay-loss trade-off？

SQ5 : Generalization / robustness

在不同 bandwidth、delay、queue 或 cross-traffic 條件下，DRL agent 的策略是否仍具備穩定表現？

3.3 Research Hypotheses

H1

在單一瓶頸鏈路中，將 throughput、RTT 與 loss 同時納入 reward，可比只追求 throughput 的 reward 更能避免高延遲或高丟包策略。

H2

DQN 搭配離散 action 能建立可行的第一版 DRL congestion-control prototype，但未必能在所有場景全面超越 CUBIC 或 BBR。

H3

在變動頻寬或含 cross traffic 的場景中，DRL agent 可能比固定規則式 baseline 顯示出更可分析的 adaptive behavior；但這需要透過實驗驗證，而非 proposal 階段直接宣稱。

3.4 Research Contribution

本專題的貢獻應保守定位為：

1. DRL formulation contribution

將 single-bottleneck congestion control 形式化為 MDP / DRL task。

2. Specification contribution

建立可轉入 OpenSpec 的 ns-3 / ns3-gym / SB3 prototype 規格。

3. Evaluation contribution

設計 NewReno、CUBIC、BBR 與 DRL agent 的 throughput-delay-loss 比較矩陣。

4. Research continuity contribution

為未來 IPFS / P2P 類網路協定中的傳輸效率研究建立底層壅塞控制知識基礎。

4. MDP Formalization

4.1 MDP 總定義

本研究將單一瓶頸鏈路壅塞控制建模為一個 Markov Decision Process：

$$M = (S, A, P, R, \gamma)$$

符號注釋：

- M ：MDP，馬可夫決策過程。
- S ：state space，狀態空間。
- A ：action space，動作空間。
- P ：transition probability，狀態轉移機制。
- R ：reward function，回饋函數。
- γ ：discount factor，折扣因子，用來決定 agent 重視短期還是長期 reward。

白話說明：

MDP 就是把「每一步看到什麼、能做什麼、做完會發生什麼、得到多少分」全部定義清楚。這對本題很重要，因為壅塞控制不是一次性判斷，而是連續決策。

4.2 Environment

本題的 environment 定義為：

- **Simulator**：ns-3
- **Topology**：single sender → bottleneck link → receiver
- **Bottleneck capacity**：可設定固定或變動頻寬
- **Propagation delay**：可設定低延遲與高延遲情境
- **Queue / buffer**：作為壅塞與延遲形成的重要因素
- **Optional cross traffic**：作為進階情境，不是 MVP 必要條件

白話說明：

環境就是一條「水管」。sender 想送資料到 receiver，中間有一段比較窄的瓶頸。agent 要學的是，怎麼送才

不會塞爆這段瓶頸。

ns-3 官方 TCP model 支援完整雙向 TCP、connection setup / close，並支援 CUBIC、NewReno、BBR 等多種壅塞控制演算法，因此適合作為後續 baseline 模擬環境。nsnam.org

4.3 Agent

本題的 agent 是：

DRL congestion-control agent

定義：

- agent 控制的是 sender-side transmission behavior。
- agent 不控制整個網路。
- agent 不改變封包實際傳播規則。
- agent 根據 observation 選擇控制動作。
- Phase 1 僅定義 abstraction，不進入實作。

白話說明：

agent 不是網路管理員，也不是 IPFS 節點；它只是 sender 端的一個控制器，負責決定接下來傳輸要更激進、維持、還是保守。

4.4 State / Observation

Minimal state version

MVP state 建議為：

$$s_t = [T_t, RTT_t, L_t, C_t, a_{t-1}]$$

符號注釋：

- s_t ：時間 t 的 state。
- T_t ：recent throughput，近期吞吐量。
- RTT_t ：round-trip time，往返延遲。
- L_t ：packet loss signal，封包遺失訊號。
- C_t ：congestion indicator，壅塞指標，可由 queue、delay growth 或 loss 近似。
- a_{t-1} ：previous action，上一個動作。

白話說明：

最小版 state 就是讓 agent 知道：最近傳多快、延遲多高、有沒有丟包、是否壅塞、上一輪做了什麼。

Enhanced state version

後續可擴充為：

$$s_t = [T_t, RTT_t, \Delta RTT_t, L_t, Q_t, B_t, a_{t-1}, \bar{T}_k, \bar{RTT}_k]$$

補充符號：

- ΔRTT_t ：RTT 變化量。
- Q_t ：queue occupancy，佇列佔用。
- B_t ：estimated bottleneck bandwidth，估計瓶頸頻寬。
- $\bar{T}_k$ ：過去 k 個時間窗的平均吞吐量。
- $\bar{RTT}_k$ ：過去 k 個時間窗的平均 RTT。

為什麼先用 minimal state 比較安全

Minimal state 較安全，因為 Phase 1 目標是 proposal 與 MVP 規格，不是追求最完整網路觀測。state 太複雜會增加 ns3-gym 串接與 Antigravity 實作風險；先用少量、可解釋、能和 reward 對應的 state，較容易完成 baseline comparison 與 DQN prototype。

4.5 Action

MVP action

建議採用離散 action：

$$A = \{decrease, keep, increase\}$$

對應白話：

- **decrease**：降低 sending rate / cwnd-like control
- **keep**：維持目前控制強度
- **increase**：提高 sending rate / cwnd-like control

為什麼第一階段不用 continuous action

continuous action 雖然更接近實際控制，但會增加：

- action scaling 風險
- reward tuning 難度
- 訓練不穩定風險
- ns3-gym interface 設計成本

因此第一階段使用離散 action，比較適合先完成 MVP。

為什麼 DQN 對此較適合

DQN 是針對離散 action 常用的 value-based DRL 方法。Stable-Baselines3 DQN 文件的 `action_space` 參數明確對應 Discrete action space，因此若本題 action 設計為 decrease / keep / increase，DQN 是 Phase 1 MVP



最合理的第一候選。 baselines3.readthedocs.io

PPO 何時才應該導入

PPO 適合在以下情況導入：

- DQN MVP 已完成
- 想改成 continuous sending rate control
- 想比較 policy-gradient 方法
- 想做更穩定但較複雜的 actor-critic 類擴充

SB3 PPO 文件說明 PPO 使用 clipping 避免 policy update 過大，適合作為後續較完整 policy optimization 路



線，但不應成為 Phase 1 第一承諾。 baselines3.readthedocs.io

4.6 Reward

基本 reward：

$$r_t = \alpha \cdot T_t - \beta \cdot RTT_t - \gamma \cdot L_t$$

符號注釋：

- r_t ：時間 t 的 reward。
- T_t ：throughput，吞吐量。
- RTT_t ：round-trip time，往返延遲。
- L_t ：loss rate，丟包率。
- α ：吞吐量權重。
- β ：延遲懲罰權重。
- γ ：丟包懲罰權重。

白話說明：

傳得越快分數越高，但如果延遲變大或丟包變多，就會扣分。

為什麼這樣能避免 agent 只追 throughput

如果 reward 只看 throughput，agent 可能學到「一直加速傳送」，但這會造成 queue 累積、RTT 變高，甚至丟包。因此 reward 必須同時放入 RTT 與 loss 懲罰，才能鼓勵比較健康的壅塞控制策略。Aurora 之所以能支撐本題，是因為它把 congestion control 視為 learning-based control problem，而不是單純 throughput maximization。 ACM Digital Library

Reward normalization 問題

不同 metric 尺度不同：

- throughput 可能是 Mbps
- RTT 可能是 ms
- loss rate 可能是 0-1

若不正規化，某個指標可能主導 reward。Proposal 階段應明確寫出：後續 implementation 需將 throughput、RTT、loss 做 scale normalization 或轉成 utility units。

Possible ablation

可規劃兩組 reward ablation：

1. Throughput-only reward

$$r_t = T_t$$

2. Trade-off reward

$$r_t = \alpha T_t - \beta RTT_t - \gamma L_t$$

比較目的：驗證 delay / loss penalty 是否能避免高吞吐但高延遲的策略。

4.7 Transition

Transition 可寫為：

$$P(s_{t+1} \mid s_t, a_t)$$

符號注釋：

- P ：狀態轉移機制。
- s_t ：目前狀態。
- a_t ：目前動作。
- s_{t+1} ：下一個狀態。

白話說明：

agent 做出一個傳輸控制動作後，下一個網路狀態由 ns-3 模擬器中的 network dynamics 決定。agent 不能直接控制封包在網路裡怎麼傳，只能影響 sender 的傳輸行為。

4.8 Policy / Objective

Policy 定義：

$$\pi(a | s)$$

符號注釋：

- π ：policy，策略。
- a ：action，動作。
- s ：state，狀態。
- $\pi(a | s)$ ：在狀態 s 下選擇動作 a 的機率或規則。

訓練目標：

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

符號注釋：

- $J(\pi)$ ：策略 π 的長期表現。
- $\mathbb{E}_{\pi}$ ：依照策略 π 互動時的期望值。
- T ：episode 長度。
- γ ：折扣因子。
- r_t ：第 t 步 reward。

白話說明：

agent 不是只追求下一秒的分數，而是要學一個長期策略：在整段傳輸過程中，讓 throughput、delay、loss 的整體表現更合理。

5. Baseline / Metrics / Experiment Matrix

5.1 Baseline

必做 baseline

1. NewReno

經典 loss-based TCP congestion control，適合作為傳統 baseline。

2. CUBIC

現代常見 TCP congestion control；ns-3 官方文件也標示 CUBIC 是 native ns-3 TCP model 支援的演算法之一，且在文件中被列為 default。  nsnam.org

強烈建議 baseline

3. BBR

BBR 以 bottleneck bandwidth 與 RTT 估計為核心，能形成和 loss-based TCP 不同的比較視角。ns-3 官方 TCP 文件列出 BBR 為支援演算法之一，因此適合列入 proposal baseline。  nsnam.org

加分 baseline

- TCP Vegas
- TCP Westwood
- DCTCP
- LEDBAT

加分 baseline 不應放入 MVP 必做，避免 scope 爆炸。

5.2 Metrics

Metric	定義	本題意義
Average throughput	單位時間內成功傳輸的資料量	衡量傳輸效率
Average RTT	封包往返時間平均值	衡量延遲與 queue 累積
Packet loss rate	丟包比例	衡量壅塞嚴重程度
Utility	綜合 throughput、RTT、loss 的分數	衡量整體 trade-off
Training reward curve	訓練過程 reward 變化	觀察 agent 是否學到穩定策略
Convergence behavior	reward / metric 是否趨於穩定	評估訓練穩定性

5.3 Experiment Matrix

Scenario ID	Bandwidth	Propagation delay	Queue setting	Traffic pattern	Baseline methods	Expected observation	Risk level
S1 Stable low-delay bottleneck	固定，例如 10 Mbps	低，例如 10 ms	中等 queue	單一長流	NewReno / CUBIC / BBR / DQN	baseline 應表現穩定，DQN 應能完成基本學習	Low
S2 Stable high-delay bottleneck	固定，例如 10 Mbps	高，例如 100 ms	中等 queue	單一長流	NewReno / CUBIC / BBR / DQN	RTT 成本變明顯，可觀察 reward 是否避免高延遲	Medium
S3 Variable bandwidth bottleneck	變動，例如 5–20 Mbps	中等	中等 queue	單一長流	NewReno / CUBIC / BBR / DQN	測試 agent 對變動容量的適應性	Medium
S4 Bottleneck with cross traffic	固定或變動	中等	中等或偏小 queue	主流 + cross traffic	NewReno / CUBIC / BBR / DQN	測試策略在干擾下的穩定性	High

這四組是 proposal 階段的 planned experiment matrix，不代表已經完成實驗。

5.4 Evaluation Philosophy

本專題成功標準不是「DRL 全面打敗所有 TCP baseline」。更合理成功標準是：

1. 建立可重現的單一瓶頸鏈路模擬規格。
2. 完成 NewReno / CUBIC / BBR baseline comparison plan。
3. 建立清楚的 DRL MDP formulation。
4. 用 throughput、RTT、loss、utility 分析 trade-off。
5. 若未來實驗發現 DQN 低於 BBR 或 CUBIC，只要分析完整，仍可成為合理 finding。

Pantheon 的價值在於 benchmark 與 reproducibility 精神，而非本專題 Phase 1 必須完整導入其工具鏈。Pantheon 論文定位是為 congestion-control schemes 提供 common interface 與 evaluation platform，這可作為本專題 benchmark philosophy 的依據。  [ACM Digital Library](#)

6. Related Work 定位

6.1 傳統 TCP congestion control

本題將 NewReno、CUBIC、BBR 定位為 baseline，而不是研究主角。NewReno 代表經典 loss-based control；CUBIC 代表現代常見 TCP 控制方法；BBR 則代表以 bottleneck bandwidth / RTT 估計為核心的不同設計思路。ns-3 官方 TCP model 支援這些 congestion control 演算法，因此它們適合作為後續可重現模擬 baseline。  [nsnam.org](#)

6.2 Learning-based congestion control

Aurora 是本題最重要的 learning-based related work。它提出用 deep reinforcement learning 來產生 congestion control policy，支撐本研究「DRL for congestion control」不是硬湊，而是已有正式研究路線。本專題不嘗試完整複製 Aurora，而是在課程 final project 範圍中，收斂成單一瓶頸鏈路、離散 action、DQN MVP 的可完成版本。  [ACM Digital Library](#)

6.3 Network simulation and RL environment

ns-3 是本專題後續模擬器基礎；ns3-gym 是 ns-3 與 RL agent 的介面橋樑。ns3-gym 官方說明其框架整合 OpenAI Gym 與 ns-3，鼓勵 RL in networking research，因此適合本題後續將 ns-3 場景包成 RL environment。  [GitHub+1](#)

6.4 Benchmark and reproducibility

Pantheon 的定位是 congestion control benchmark 與 reproducibility 的參考。它不應作為 Phase 1 必裝依賴，因為本專題最低風險路線是先完成 ns-3 baseline 與 ns3-gym MVP；但 Pantheon 的 common interface、baseline comparison 與 reproducibility 思想，應被寫進 Related Work / Benchmark Philosophy。  [ACM Digital Library](#)

7. 風險清單

Risk ID	Risk description	Probability	Impact	Prevention	Fallback	影響 proposal	影響 final implementation
R1	ns3-gym 安裝 / 版本問題	高	高	Phase 2 前先固定 OS、ns-3、Python 版本；OpenSpec 明確列 environment	改用簡化 Python Gym bottleneck environment 作 fallback	中	高
R2	ns-3 baseline logging 不完整	中	高	baseline change 先做 logging spec：throughput、RTT、loss	先只做 throughput + RTT, loss 作加分	中	高
R3	DQN 不收斂	中	高	不承諾全面 outperform；先用少量 state / 離散 action	改成 random / heuristic / trained DQN behavior analysis	低	高
R4	Reward 設計導致高吞吐但高 RTT / loss	高	高	reward 需包含 RTT / loss penalty	做 ablation，比較 throughput-only vs trade-off reward	低	高
R5	BBR 在 ns-3 中跑法或版本相依性問題	中	中	先查 ns-3 版本與 TCP model 支援	MVP baseline 只保留 NewReno + CUBIC, BBR 改強烈建議	低	中
R6	Antigravity 實作偏航	中	高	所有任務先轉 OpenSpec change, 逐項驗收	每次只允許執行單一 change, 不允許自由擴題	中	高
R7	題目被誤解成 IPFS 實作	中	中	proposal 明確寫：IPFS 是動機延伸，不是本題 scope	在標題與摘要避免放 IPFS 主體詞	高	中
R8	實驗結果未超越 baseline	高	中	evaluation philosophy 明確寫不以全面打敗 baseline 為唯一成功標準	改成 trade-off analysis / limitation / expected future work	低	中

Risk ID	Risk description	Probability	Impact	Prevention	Fallback	影響 proposal	影響 final implementation
R9	action 無法順利接到 ns-3 傳輸控制	中	高	Phase 1 僅定義 abstraction, Phase 2 先做 smoke test	改成外部 rate control abstraction 或 simplified control loop	低	高
R10	10 分鐘影片太技術、老師不易懂	中	中	PPT 放 topology、MDP、metric 三張核心圖	用 bottleneck 水管比喻補強口說	中	低

8. OpenSpec 對應表

Phase 1 artifact	對應 OpenSpec change	用途	Required before coding	可後補
正式題目與定位	01-project-charter	凍結研究方向與 non-goals	是	否
A-F Abstract	01-project-charter	定義 proposal 敘事與研究價值	是	可小修
Research Questions	01-project-charter	定義研究問題與成功標準	是	可小修
Research Hypotheses	01-project-charter	防止後續過度承諾	是	可小修
MDP Formalization	03-ns3-gym-environment	定義 state/action/reward/environment	是	否
Minimal / Enhanced State	03-ns3-gym-environment	指導 observation space 設計	是	Enhanced 可後補
Action Design	03-ns3-gym-environment / 04-dqn-mvp-agent	決定 DQN 是否可行	是	否
Reward Design	03-ns3-gym-environment / 04-dqn-mvp-agent	指導 reward function	是	可調參
Baseline List	02-ns3-baseline-benchmark	指導 TCP baseline 實驗	是	加分 baseline 可後補
Metrics	02-ns3-baseline-benchmark / 04-dqn-mvp-agent	指導 logging 與圖表	是	否
Experiment Matrix	02-ns3-baseline-benchmark / 04-dqn-mvp-agent	指導實驗配置	是	可逐步擴展
Risk Register	01-project-charter	範圍管理與降階策略	是	可更新
Proposal Skeleton	05-reporting-figures-and-demo	後續報告與 PPT 骨架	否	是

Phase 1 artifact	對應 OpenSpec change	用途	Required before coding	可後補
10 分鐘簡報骨架	05-reporting-figures-and-demo	後續影片 / PPT	否	是

OpenSpec 官方文件指出，每個 change 可集中管理 proposal、design、tasks、specs，並在 implementation 前先建立對齊文件；因此這張表就是後續 Antigravity 工作前的上游索引。 [GitHub+1](#)

9. Proposal Skeleton

1. Title

Section purpose：快速讓老師知道題目範圍。

要放什麼：中文題目、英文題目、作者、課程、日期。

不要放什麼：不要把 IPFS 放成主題。

建議圖表：簡單 bottleneck topology 圖。

2. Abstract

Section purpose：用 A-F 完整說服。

要放什麼：Attention / But / Cure / Development / Experiments / Findings。

不要放什麼：不要宣稱已完成實驗。

建議圖表：可不放，或放一張 A-F flow。

3. Introduction / Motivation

Section purpose：說明為什麼網路吞吐量與壅塞控制重要。

要放什麼：分散式系統、內容傳輸、P2P / IPFS 背景連結、但範圍收斂到 single bottleneck。

不要放什麼：不要深入 IPFS 協定細節。

建議圖表：分散式傳輸 → bottleneck link 的示意圖。

4. Problem Statement

Section purpose：把問題從「網路很重要」收斂成「DRL 可以處理的序列決策問題」。

要放什麼：throughput、RTT、loss trade-off；traditional TCP heuristic 的限制。

不要放什麼：不要亂批傳統 TCP 無效。

建議圖表：throughput vs RTT / loss trade-off 三角圖。

5. Related Work

Section purpose：定位本題站在誰的肩膀上。

要放什麼：NewReno / CUBIC / BBR、Aurora、ns3-gym、Pantheon。

不要放什麼：不要寫成超長 survey。

建議圖表：Related Work map。

6. Proposed Design

Section purpose：寫成規格書。

要放什麼：ns-3、ns3-gym、SB3、DQN、baseline、metrics。

不要放什麼：不要寫 code-level implementation。

建議圖表：system architecture diagram。

7. MDP Formulation

Section purpose：證明這是正當 DRL 題。

要放什麼： $M = (S, A, P, R, \gamma)$ 、state、action、reward、policy。

不要放什麼：不要用過度複雜 state。

建議圖表：agent-environment loop。

8. Experimental Plan

Section purpose：說明未來怎麼驗證。

要放什麼：baseline、metrics、scenario matrix。

不要放什麼：不要放假結果。

建議圖表：experiment matrix table。

9. Risk and Scope Control

Section purpose：證明你知道怎麼把專案做完。

要放什麼：risk register、fallback、not-do list。

不要放什麼：不要把風險藏起來。

建議圖表：risk matrix。

10. Expected Contributions

Section purpose：說明本專題可能貢獻。

要放什麼：DRL formulation、可重現規格、baseline comparison、IPFS future connection。

不要放什麼：不要說已經顯著超越 baseline。

建議圖表：contribution bullets。

11. Roadmap

Section purpose：連接 Phase 2 / OpenSpec / Antigravity。

要放什麼：01-project-charter → 02-baseline → 03-env → 04-DQN → 05-reporting。

不要放什麼：不要寫成實作細節清單。

建議圖表：timeline / phase roadmap。

這個骨架繼承你提供的 oral defense 範例中的 Motivation、Related Works、Proposed Method、Experimental Results 節奏，但本階段要改寫成 proposal，因此 Experimental Results 要轉成 Experimental Plan / Expected Findings。

Frank_oral_defense_0725_v7

10. 10 分鐘簡報骨架

Slide	Slide title	Key message	Suggested visual	Speaker note direction	對應
1	Title	本題是 DRL-based congestion control over bottleneck link	題目 + bottleneck topology	先講這不是 IPFS 實作，而是底層傳輸控制研究	Title
2	Why Throughput Matters	分散式與 P2P 系統依賴穩定傳輸	分散式系統 / P2P / bottleneck 示意	連到你畢論背景，但收斂到基礎問題	Attention
3	The Congestion-Control Trade-off	throughput、RTT、loss 互相拉扯	三角 trade-off 圖	只追 throughput 會造成 delay / loss	But
4	Limitations of Rule-Based TCP Control	傳統 TCP 很成熟，但多為 rule-based / heuristic	NewReno / CUBIC / BBR 對照	精準批判，不要貶低 TCP	But / Related Work
5	Proposed Cure: DRL Formulation	把 congestion control 改寫成 sequential decision problem	agent-environment loop	agent 觀測網路狀態並選控制動作	Cure
6	System Design	ns-3 + ns3-gym + SB3	system architecture	說清楚工具鏈怎麼串	Development
7	MDP Formulation	S/A/R/P/γ 成立	MDP 表格或流程圖	用白話解釋 state / action / reward	Proposed Design
8	Baselines and Metrics	比 NewReno、CUBIC、BBR；看 throughput、RTT、loss、utility	baseline / metrics table	不承諾全面打敗，只做公平比較	Experiments
9	Experiment Matrix	四組 planned scenarios	matrix table	說明低延遲、高延遲、變動頻寬、cross traffic	Experiments
10	Expected Findings	目標是 trade-off analysis，不是假結果	expected outcome diagram	強調 proposal 階段不宣稱結果	Findings
11	Risk and Scope Control	題目能做完的關鍵是守住範圍	risk matrix	不做 IPFS、QUIC、kernel-level	Risk
12	Roadmap to Implementation	後續交給 Antigravity，以 OpenSpec 推進	OpenSpec changes roadmap	Phase 2 才開始 baseline，不直接跳 DQN	Roadmap

11. Phase 1 最終交付清單

11.1 本次 Phase 1 已完成的文件項目

本次已完成：

- 正式題目與研究定位
- A-F Abstract
- Main Research Question
- Sub-questions
- Research Hypotheses
- Research Contribution
- MDP Formalization
- State / Action / Reward proposal
- Baseline / Metrics / Experiment Matrix
- Related Work 定位
- Risk Register
- OpenSpec 對應表
- Proposal Skeleton
- 10 分鐘簡報骨架

11.2 下一階段 Phase 2 前必須凍結的事項

Phase 2 前必須凍結：

1. 使用哪個 ns-3 版本。
2. 是否能在目標環境安裝 ns3-gym。
3. Baseline 必做清單是否為 NewReno + CUBIC，BBR 是否列為強烈建議。
4. Minimal state 是否採用 $[T_t, RTT_t, L_t, C_t, a_{t-1}]$ 。
5. MVP action 是否固定為 decrease / keep / increase。
6. 第一版 reward 權重是否先採等權或需正規化後再定。

11.3 哪些內容可直接轉入 OpenSpec

可直接轉入：

- 01-project-charter：題目、定位、non-goals、風險清單、evaluation philosophy
- 02-ns3-baseline-benchmark：baseline、metrics、experiment matrix
- 03-ns3-gym-environment：MDP、state、action、reward、transition
- 04-dqn-mvp-agent：DQN MVP、reward curve、evaluation plan
- 05-reporting-figures-and-demo：PPT skeleton、figures list、demo flow

11.4 哪些內容可直接轉入 PPT

可直接轉入：

- Slide 1：正式題目
- Slide 2-3：Attention / But
- Slide 4：Related Work baseline
- Slide 5-7：Cure / Development / MDP
- Slide 8-9：Baseline / Metrics / Experiment Matrix
- Slide 10：Expected Findings
- Slide 11：Risk Control
- Slide 12：Roadmap

11.5 哪些內容可直接轉入 GitHub README

README 可直接放入：

- Project title
- One-sentence thesis
- Scope / Non-goals
- Planned toolchain
- Baselines
- Metrics
- Experiment scenarios
- OpenSpec workflow
- Current phase：Proposal only, no implementation yet

Phase 1 結論

這份 Phase 1 proposal 主體已經可以作為後續 OpenSpec 與 Antigravity 的上游基礎。下一步不應直接寫 code，而是先產生 **OpenSpec change 01：project-charter**，把本次 proposal 中的題目、scope、non-goals、研究問題、風險與成功標準寫成可審核的規格文件。

Phase 5：做 DQN MVP

截圖是 Phase 4：DRL MVP Implementation 的 commit

Antigravity 做完「Phase 4：DRL MVP Implementation」 ****最佳 Prompt**** 指令了

並已推上 github Phase 4：DRL MVP Implementation 的異動 請幫我驗收 git diff

是否全部 ****Phase 4：DRL MVP Implementation**** 要求都有通過

是否能推進到「****Phase 5：做 DQN MVP****」了呢

顯示更多顯示較少

結論：**OpenSpec Change 05：reporting-figures-and-demo** 可以驗收通過，可以推進到「**Phase 5：Final Reporting / Demo / PPT Package**」。

但我會把這次驗收判定為：

規格內容通過；流程佐證與 Markdown 可讀性有小瑕疵，但不構成 blocker。

這個 commit 確實新增了 `openspec/changes/reporting-figures-and-demo/`，包含 `proposal.md`、`design.md`、`tasks.md` 與多個 `specs/` 檔案，共 **13 files changed**、**633 additions**、**0 deletions**，而且沒有修改 Phase 3 / Phase 4 的實驗數據，符合 Change 05「只做 final reporting / demo / PPT 規格，不新增實驗、不重訓、不改數據」的定位。  [GitHub](#)

1. 這次 commit 做了什麼

這次 commit 新增的檔案結構基本符合我們前面要求的 Change 05 規格包：

```
openspec/changes/reporting-figures-and-demo/
  .openspec.yaml
  proposal.md
  design.md
  tasks.md
  specs/
    artifact-manifest.md
    demo-script.md
    figure-package.md
    final-report-package.md
    ppt-package.md
    readme-finalization.md
    result-interpretation.md
    risk-register.md
    reporting-figures-and-demo/spec.md
```

GitHub diff 顯示新增的 file tree 中確實包含以上核心檔案，尤其是 `proposal.md`、`design.md`、`tasks.md`、`specs/artifact-manifest.md`、`specs/demo-script.md`、`specs/figure-package.md`、`specs/final-report-`

`package.md`、`specs/ppt-package.md`、`specs/readme-finalization.md`、`specs/result-interpretation.md`、`specs/risk-register.md`。 [GitHub](#)

這和官方 OpenSpec 的 change folder 模式一致：官方 README 的示例是 `/opsx:propose` 會建立 `openspec/changes/<change-id>/`，內含 `proposal.md`、`specs/`、`design.md`、`tasks.md`；官方也說 OpenSpec 的目的就是讓人與 AI 在寫 code 前先對齊規格。 [GitHub](#)

2. Change 05 核心要求驗收

驗收項目	結果	判斷
建立 reporting-figures-and-demo change folder	有	✓ 通過
proposal.md	有，且說明 Why / What Changes / What Does Not Change / Impact / Dependencies / Acceptance Criteria	✓ 通過
design.md	有，且包含 final narrative、figure、demo、PPT、governance design	✓ 通過
tasks.md	有，包含 OpenSpec verification、artifact audit、report、figure、demo、PPT、README、manifest、risk、validation 任務	✓ 通過，但格式需小修
final report spec	有 final-report-package.md ，定義 14 節 final report 結構	✓ 通過
figure package spec	有 figure-package.md ，定義 9 張 final figures、source mapping、PPT-ready 規則	✓ 通過
demo script spec	有 demo-script.md ，定義 10 分鐘 demo flow、commands、no-go statements	✓ 通過
PPT package spec	有 ppt-package.md ，定義 10–12 頁簡報結構與每頁重點	✓ 通過
README finalization spec	有 readme-finalization.md ，要求清理 TODO、Phase 4 pending 舊文字、補 final artifact links	✓ 通過
artifact manifest spec	有 artifact-manifest.md ，定義 9 類 artifact 與 table schema	✓ 通過
result interpretation rules	有，明確規範 S1 / S2 解讀、delay proxy、Fallback Option B、不可 overclaim	✓ 通過
risk register	有，列出 14 項 reporting / figure / demo / overclaiming 風險	✓ 通過
不重訓 / 不新增實驗 / 不改數據	proposal 明確寫 no baseline changes、no DQN model/training changes、no new experiments	✓ 通過
不加入 PPO / IPFS / QUIC / multi-agent / multi-path	proposal 與 tasks 都明確禁止	✓ 通過
是否能支撐 Phase 5 implementation	可以	✓ 通過

3. 規格內容是否符合「高分交付物封裝」方向

符合，而且方向很準。

design.md 已把 final narrative 收束成很適合期末報告的主線：傳統 TCP 是 rule-based、專題把 single bottleneck congestion control 建模為 DRL task、Phase 3 做 NewReno / CUBIC / BBR baseline、Phase 4 做

ns3-gym + SB3 DQN MVP、最後誠實呈現 DQN S1 utility 排第 2、S2 utility 排第 3，不宣稱 DRL 全面勝過 TCP。 [GitHub](#)

這點非常重要，因為 Phase 5 的任務不是再做更多實驗，而是把成果包裝成老師看得懂、能快速驗收、也能展示研究價值的高分交付物。這次 Change 05 已經把「成果封裝」所需的 final report、figures、demo、PPT、README、artifact manifest、interpretation rules 全部納入規格。

4. 最重要的通過點：沒有過度宣稱

這次我最在意的是 Antigravity 有沒有開始亂吹 DQN 成果。看起來沒有。

result-interpretation.md 明確規定：

- S1：DQN utility 0.900，排名第 2，贏 CUBIC / NewReno，但沒有贏 BBR。
- S1 的 100% increase action 要解讀成 benign / near-capacity environment 下的局部策略，不可誇大為複雜 adaptive behavior。
- S2：DQN throughput 高，但 loss rate 5.54%，utility 排第 3，輸給 NewReno / CUBIC。
- delay 是 proxy，不是 true RTT。
- DQN 是 sender-side rate-control abstraction，不是 kernel-level cwnd control。 [GitHub](#)

這完全符合我們前面對 Phase 5 的要求：**高分不是靠誇大結果，而是靠誠實、可重現、能分析限制。**

5. Figure / Demo / PPT 規格是否足夠支撐 Phase 5

足夠。

figure-package.md 已定義 9 張 final figures，包括 system pipeline、single bottleneck topology、MDP formulation、baseline utility summary、DQN reward curves、action distribution、DQN vs baseline utility、DQN vs baseline loss、key findings summary；也要求圖表必須有 title、axis labels、scenario、units，並明確標示 delay proxy 與 provisional utility。 [GitHub](#)

demo-script.md 已把 10 分鐘 demo 拆成 10 段，包含 project overview、OpenSpec SDD proof、Phase 3 baseline artifacts、Phase 4 ns3-gym + DQN artifacts、smoke test、DQN training/evaluation、DQN vs baseline comparison、honest limitations、reproduce instructions、closing statement；也明確禁止假裝 live training、禁止宣稱 DQN 全面勝過 baseline、禁止隱藏 S2 high loss、Fallback Option B、delay proxy。 [GitHub](#)

ppt-package.md 也已定義 12 頁簡報，包括 Title、Motivation、Research Question & Scope、System Architecture、MDP Formulation、Baseline Benchmark、DRL MVP Implementation、S1 Results、S2 Results、Findings / Limitations、Demo / Reproducibility、Conclusion / Future Work。 [GitHub](#)

這些都能直接進入下一階段的 Phase 5 implementation。

6. 需要注意的小瑕疵

我看到三個小問題，但都不是 blocker。

A. tasks.md 的勾選狀態全部還是未勾

tasks.md 已有完整任務清單，但全部是 []。這在「change proposal / spec 階段」可以接受，因為它本來就是規格任務清單；但如果 Antigravity 聲稱「Change 05 完成」，理想上至少應該在提交訊息或 completion report 中說明這些 tasks 是待 Spec Owner 驗收後再勾，或在下一個 commit 補勾完成項目。 [GitHub](#)

建議： 你可以接受它目前狀態，但 Phase 5 implementation 開始前，請 Antigravity 先補一個小 commit，將 Change 05 spec 任務完成項目勾選，或新增 **Change 05 Completion Report**。

B. 部分 raw markdown 換行品質不漂亮

raw **tasks.md** 與某些 specs 看起來有不少內容被壓在同一行，這不一定會影響內容，但會影響後續 code review / diff readability。 [GitHub+1](#)

建議： 非 blocker，但 Phase 5 implementation 前可以讓 Antigravity 做一次 markdown formatting cleanup，讓每個 heading、list、table 都正常換行。

C. 官方 OpenSpec 執行證據仍只看到產物，不一定看到 command output

commit 中有 **.openspec.yaml**，內容顯示 **schema: spec-driven** 與建立日期；也有符合 OpenSpec change folder 的 proposal / design / tasks / specs 結構。 [GitHub+1](#)

但單看 GitHub commit，我無法直接證明 Antigravity 當時真的執行了 **openspec validate reporting-figures-and-demo --strict**，除非它有把 command output 放在 completion report 或 commit message 裡。官方 OpenSpec README 的 quick start / workflow 確實要求安裝、初始化、使用 **/opsx:propose** 並透過 change folder 管理 proposal、specs、design、tasks。 [GitHub](#)

建議： 這不是內容 blocker，但你可以要求 Phase 5 implementation prompt 一開始先補：

```
Run openspec validate reporting-figures-and-demo --strict and paste result into
reports/final/change05-validation-note.md
```

7. 是否全部 Change 05 要求都有通過？

我的判定：

內容層：通過。

OpenSpec 結構層：通過。

嚴格流程佐證層：小缺口，但不阻塞。

更精準地說：

層級	判斷
規格完整性	✓ 通過
是否涵蓋 final report / figures / demo / PPT / README / manifest / interpretation / risks	✓ 通過
是否守住 no retraining / no fake result / no PPO / no scope expansion	✓ 通過
是否能作為 Phase 5 implementation 的 source of truth	✓ 通過

層級	判斷
是否有完整 command output 證明 validate passed	⚠️ GitHub diff 中沒看到
Markdown diff 可讀性	⚠️ 可再整理

所以我不會擋你進下一階段。

8. 是否能推進到 Phase 5 ?

可以。

但我建議把下一階段命名再收斂得更清楚一點：

Phase 5 : Final Reporting / Demo / PPT Package Implementation

或如果你想沿用你提到的高分交付物語感，也可以叫：

Phase 5 : 把成果封裝成高分交付物

下一階段的任務才是正式產出：

- reports/final/final-report.md
- figures/final/*.png
- demo/demo-script.md
- demo/demo-checklist.md
- slides/final/final-presentation-outline.md
- slides/final/speaker-notes.md
- slides/final/slide-assets.md
- reports/final/final-artifact-manifest.md
- README final update

9. 我的最終驗收判定

OpenSpec Change 05 : reporting-figures-and-demo — 驗收通過。

等級：

Minimum Acceptance :  通過

Strong Acceptance :  通過

Excellent Acceptance :  接近通過，但建議補 validate note / markdown cleanup

不需要重做 Change 05，也不需要退回 Phase 4。

下一步可以正式進入：

Phase 5 : Final Reporting / Demo / PPT Package Implementation

我建議你下一個 prompt 給 Antigravity 時，第一段先要求它補做兩件小事：

Before Phase 5 implementation, run OpenSpec strict validation for reporting-figures-and-demo and record the result. Then perform markdown formatting cleanup if needed, without changing the specification content.

補完後，就可以開始正式封裝期末專題高分交付物。

6. 需要注意的小瑕疵

**唯一需求**

我要請 Antigravity 幫我先修好瑕疵再****真正進入 Phase 5 實作****

**必備前置需求**

另外 Antigravity AI Agent 服務 因為我在 Agent 模型用 Claude 實作 但 Claude model 額度用完了 現在打算改用 Gemini Pro model 我要走下面這個流程

最佳流程不是「直接把模型從 Claude 換成 Gemini 後叫它繼續」，而是：

先請 Claude 產生一份正式交接文件 → 再讓 Gemini 讀整個 repo / 資料夾 / git diff / Antigravity Artifacts / Claude 交接文件 → Gemini 先輸出接手理解報告 → 你確認後才讓它繼續改 code。

這樣後續 Gemini 的表現通常會更穩，因為它不是在「猜 Claude 前面做了什麼」，而是用一份結構化交接文件重新建立任務狀態。

所以

我要你先幫我生成一個餵給 Antigravity Claude model 的最佳 Prompt 讓他無死角的生成交接事項

可以讓 Antigravity Gemini model 無縫銜接我的 DRL 期末專題報告

等到交接事項由 Claude model 製作完畢後 代表兩個 model 的理解****對齊****了

****對齊****了之後才真正開始先修好「OpenSpec Change 05：reporting-figures-and-dem」的瑕疵

最後才進到下一個階段「Phase 5：Final Reporting / Demo / PPT Package Implementation」

現在

請直接開始生成餵給 ****Antigravity Claude model**** 處理兩模型交接事項的最精準執行任務的「最完美且超高品質的****最佳 Prompt****」

顯示更多顯示較少

我先把定位講清楚：這份 prompt 的任務**不是讓 Claude 修瑕疵，也不是進 Phase 5**，而是讓 Claude 作為「上一任實作者」產出一份可被 Gemini 接手的正式交接文件。OpenSpec 官方定位是用 spec-driven workflow 讓人與 coding agent 在實作前對齊規格；官方文件也明確描述標準流程是 **proposal → specs → design → tasks**

→ **implement**，所以交接文件要把 repo 狀態、已驗收 changes、未修瑕疵與下一步入口全部整理成新的 source-of-truth 補充材料。  OpenSpec+1

下面這段可以直接貼給 **Antigravity Claude model**。

這份 prompt 的重點是把 Claude 的任務鎖死在「正式交接」，不要讓它順手修 Change 05，更不要讓它偷跑 Phase 5。等 Claude 產出交接文件後，下一步才是讓 Gemini 先輸出接手理解報告。

Phase 7：把成果封裝成高分交付物

<https://github.com/donuop35/ns3-drl-congestion-control/commits>

我已完成「Phase 5 如何實作的過程」並且我驗收完成 上方這個 GitHub Repo 是「Phase 5 如何實作的過程」目前的成果 接下來我要正式進入「Phase 7：把成果封裝成高分交付物」

我在**「[DRL][FR] Final Proposal Phase 5」交談記錄** 畫面左側也都有完整過程 請去詳細閱讀理解「**Phase 5 如何實作的過程**」這個階段的實作過程

請幫我生成可以讓 Antigravity 最精準執行「Phase 7：把成果封裝成高分交付物」任務的正式「最完美且超高品質的**最佳 Prompt**」

請記得

此份「最完美且超高品質的**最佳 Prompt**」是要給 Antigravity **Claude** 看的 而非 Antigravity Gemini 請**務必嚴格遵守 Claude 開發架構模式**
顯示更多顯示較少

我先說明依據：我無法直接讀取你左側「[DRL][FR] Final Proposal Phase 5」完整聊天畫面，但我已依你提供的 GitHub repo 現況補足判斷。commit history 顯示 Phase 5 已有「implement final reporting demo and slide package」「polish final package」「fix final figures QA」等提交；README 也標示 **Phase 5: Final Package Complete / Pending Spec Owner Final Review**，並列出 Final Report、Final Figures、Demo Script、Slides、Artifact Manifest、Reproducibility Guide 等成果。 

目前 final report 已誠實呈現 S1/S2 結果、DQN 限制、delay proxy、Fallback Option B、S2 high loss 與「不宣稱全面勝過 TCP」；submission checklist 也標示 no fake results、no overclaim、no frozen data modified、no retraining 等規則已納入。 

因此 Phase 7 的定位不應再做新實驗，而應是：**把既有成果整理成老師一看就懂、GitHub 可快速驗收、簡報/影片可直接使用的高分交付物封裝層。**

下面這份可以直接貼給 **Antigravity Claude**。

這份 prompt 已經把 Phase 7 鎖定為「封裝與高分交付物」，不是新實驗。它也把 Claude 的工作方式明確化為 **Explore → Plan → Implement → Verify → Commit → Report**，避免 Claude 直接大改或誤把 Phase 7 做成新的研究階段。